Day 1: 24-07-26 Friday (online)

==========

Institute Name : RaiseTech

Website : www.raisetech.in

Location : Ameerpet, Hyderabad

Course Name : Full Stack Java with GenAI

Duration : 5 Months

Mode : Online/Offline

Additional classes : DSA , AWS, AI

Recording videos : Free

Batch No : RT-JAVA-064

Project : Employee Management System

Trainer Name : Niyaz Sir

Experience : 16+ years of experience in development and training.

Full Stack Java

|

|----------|

Frontend Technologies Backend Technologies

> HTML5 > Core Java

> CSS3 > Advanced Java

> JavaScript > Oracle DB

> Bootstrap5 > Frameworks

> ReactJS > Spring Boot

> Project > Microservices

> Spring AI

> DSA

> AWS

> Project

Java

====

Object oriented programming language

Platform independent programming language

Case sensitive programming language

Strongly typed checking language

High level programming language

Open source programming language

1995 --> James Gosling --> Java (Sun Micro System)

2010 --> Oracle Corporation

Java Software --> JDK software

C

===

Procedure oriented programming language

Platform dependent programming language

Case sensitive programming language

Loosely typed checking language

Middle level language (Low + High)

Interview Question

==========

Q) What is Java?

Java is a object oriented, platform independent, case sensitive, strongly typed checking, high level, open source programming language developed by James Gosling in the year of 1995.

Day:2 25-07-26 Saturday (online)

==========

Q) What is the difference between Python and Java?

Python	Java
It is developed by Guido Van Rossum.	It is developed by James Gosling.
It is managed by Python Software	It is managed by Oracle Corporation.

Foundation (PSF).

It is often used as scripting It is often used as object oriented programming language. language.

It is a dynamically typed language. It is a statically typed language.

It is a interpreted language. It is a compiled language.

It uses PVM to execute python byte code.It uses JVM to execute java byte code.

Performance is low. Performance is high.

There is less security. It is highly secured.

AI
===
AI stands for Artificial Intelligence.
AI is a capability of a computer or a system to mimic or simulate human behaviours.
ex:
Thinking
Reasoning
Language understanding
Perception
and etc.
We have three types of AI.

1) Weak / Narrow AI

It is limited to narrow areas.
ex:
ChatGPT, Alexa, Siri, GoogleMaps and etc.

2) Strong AI

It is used to mimic human behaviours.
ex:
Not Yet Achieved

2) Super AI

It is used to surpass human behaviours.

ex:

Not Yet Achieved

Day:3 27-07-26 Monday

==========

Escape Characters / Escape Sequences

==========

Escape characters are used to design our output in neat and clean manner.

Escape characters start with back slash (\) followed by a character.

ex:

\n

Mostly escape characters are written in output statement in java.

ex:

System.out.println("\n");

We have following list of escape characters in java.

1) \n (new line)
2) \t (horizontal tab)
3) \b (back space)
4) \r (carriage return)
5) \f (form feeding)
6) \\ (back slash)
7) \" (double quote)
8) \' (single quote)

and etc.

1) \n (new line)

---

class Vihaar

```
{
public static void main(String[] args)
{
System.out.println("RAISE\nTECH");
}
}
```

o/p:

RAISE

TECH

2) \t (horizontal tab)

```
class Ganesh
{
public static void main(String[] args)
{
System.out.println("RAISE\tTECH");
}
}
```

o/p:

RAISE TECH

3) \b (back space)

```
class Rohit
{
public static void main(String[] args)
{
System.out.println("RAISE\bTECH");
}
}
```

o/p:

RAISTECH

ex:

```
class Ranil
{
```

```
public static void main(String[] args)
{
System.out.println("RAISE\b\b\bTECH");
}
}
```

o/p:

RATECH

4) \r (carriage return)

```
class Uday
{
public static void main(String[] args)
{
System.out.println("RAISE\rTECH");
}
}
```

o/p:

TECHE

ex:

```
class Uday
{
public static void main(String[] args)
{
System.out.println("TECH\rRAISE");
}
}
```

o/p:

RAISE

5) \f (form feeding)

```
class Shekhar
{
public static void main(String[] args)
{
```

```
System.out.println("I\fLove\fIndia");
}
}
```

o/p:

I

Love

India

6) \\ (back slash)

```
class Fayaz
{
public static void main(String[] args)
{
System.out.println("RAISE\\TECH");
}
}
```

o/p:

RAISE\TECH

C program

Q) Write a c program to print %d ?

```
void main()
{
clrscr();

print("%%d"); // %d

getch();
}
```

7) \" (double quote)

```
class Leela
{
public static void main(String[] args)
{
System.out.println("I love \"java\" programming");
}
}
```

o/p:

I love "java" programming

8) \' (single quote)

```
class Veera
{
public static void main(String[] args)
{
System.out.println("I love 'java' programming");
System.out.println("I love \'java\' programming");
}
}
```

o/p:

I love 'java' programming

I love 'java' programming

Screening Testing Program

==========

Q) What will be the output of below snippet?

```
class Example
{
public static void main(String[] args)
{
System.out.print("\nKO");
System.out.print("\bLI");
System.out.print("\rHA");
}
```

}
o/p:
HAI

Day:4 28-07-26 Tuesday
==========

Q) What is the difference between C++ and Java?

C++	Java
It is developed by Bjarne Stroustrup.	It is developed by James Gosling.
It is a partial object oriented programming language.	It is a fully object oriented programming language.
It is a platform dependent.	It is platform independent.
Memory allocation and deallocation will taken care by a programmer.	Memory allocation and deallocation will taken care by a JVM.
It supports multiple inheritance.	It does not support multiple inheritance.
It supports pointers.	It does not support pointers.
It supports operator overloading.	It does not support operator overloading.
It supports goto statement.	It does not support goto statement.
It supports preprocessor directory(#).	It does not support preprocessor directory.
It contains three acess specifiers i.e public, private and protected.	It contains four access modifiers i.e default,public,private and protected.
It contains three loops i.e do while loop, while loop and for loop.	It contains four loops i.e do while loop, while loop, for loop and for each loop.

To save the program we will use To save the program we will use .java
.cpp extension. extension.

C++ Program
==========
Q) How can we execute if and else simultenously?

```
#include<iostream.h>
#include<conio.h>
void main()
{
clrscr();
if(0)
{
label1 : cout<<"If Stmt";
goto label2;
}
else
{
goto label1;
label2 : cout<<"Else Stmt";
}

getch();
}
```

Q) What is the difference between .NET and Java?

.NET	Java
It is a product of Microsoft.	It is a product of Oracle Corporation.
It is a platform dependent.	It is a platform independent.
It contains less set of frameworks. ex:	It contains large set of frameworks. ex:

asp.net Hibernate
asp.net mvc spring
and etc. spring boot
microservices
spring security
spring AI
and etc.

To develop medium scale projects we To develop large scale projects we
will use .NET. will use java.

Day:5 29-07-26 wednesday
==========

SDLC
====
SDLC stands for Software Development Life Cycle.

SDLC is a process adopt by IT industries to develop accurate and quality of software.

There are six phases in SDLC.

1) Feasibility study
2) Analysis
3) Designing
4) Coding
5) Testing
6) Delivery and Maintenance

1) Feasibility study

Feasibility study completely relay on TELOS formulae.
ex:
T - Technical Feasibility
E - Economical Feasibility

L - Legal Feasibility
O - Operational Feasibility
S - Schedule Feasibility

All the above information they will keep in a document called BDD.
BDD stands for Business Design Document.

2) Analysis

In analysis phase, system analyst or product owner will involved.
They will separate system requirements and software requirements.
They will keep above information in a document called SRS.
SRS stands for Software Requirement Specification.

3) Designing

We have two types of designing.

1) High Level Designing (HLD)

In high level designing, we will design main/major modules.
A technical manager is responsible to perform high level designing.
ex:
Payment Module

2) Low Level Designing

In low level designing, we will design child/sub modules.
A team lead or project lead is responsible to perform low level designing.
ex:
Payment Initiation
Payment gateway integration
payment history
payment process
payment refund
and etc.

All the above information they will keep in a document called PDD/TDD.

PDD stands for Project Design Document.

TDD stands for Technical Design Document.

4) Coding

In coding phase, developers will involved.

They are responsible to write the code, generate the build and perform unit testing.

Checking piece of code working as per requirement or not is called unit testing.

5) Testing

In testing phase, testing team or QA team will involved.

They will use one software component called STLC(Software Testing Life Cycle) to remove all the bugs from project.

6) Delivery and maintenance

Before delivery we perform UAT testing.

UAT stands for User Acceptance Testing.

It is categories into two types.

i) Alpha Testing

ii) Beta Testing

Day:6 30-07-26 Thursday

==========

Course Name : Full Stack Java + GenAI

Duration : 5 Months

Project : Employee Management System

Mode : Online/Offline

Batch No : RT-JAVA-064
Mode : Online/Offline

==========

Comments in java
==========
Comments are created for documentation purpose.

It is highly recommended to use comments in regular programming.

Comments are used to improve readability of our code.

Diagram: class1.1

Comments won't display in output because they won't compiled by the compiler.

We have two comments in java.

1) Single Line Comment

It is used to comment a single line.
ex:
// Comment Here

2) Multi Line Comment

It is used to comment multiple lines.
ex:
/*
-
- comment here
-
*/

ex:

```
//class declaration
class Test
{
//main method
public static void main(String[] args)
{
//variable declaration
int x = 10;

//output stmt
System.out.println(x);
}
}
```

Project

=======

A project is a individual or collaborative enterprise that is carefully planned to achieve particular goal/aim.

Technically, a project is a collection of modules.

ex:

admin module

customer module

payment module

addtocart module

report generation module

shipping module

and etc.

Every project contains two domains.

1) Technical Domain

Technical domain describes using which technology , tool or language is used to develop the project.

ex:

Java

2) Functional Domain

Functional domain describes business area where our project is used to solve the problems.

ex:

Healthcare domain

Banking domain

ERP domain

Insurance domain

Types of IT companies

==========

Mainly we have two types of IT companies.

1) Product Based Company

---

ex:

Oracle, Microsoft, Google, Amazon and etc.

2) Service Based Company

---

ex:

TCS, Accenture, cognizant, Capgemini and etc.

Day:7 31-07-26 Friday

==========

Programming Language

==========

It is a set of instructions given to a computer to perform perticular task.

A language which is used to communicate between user and computer is called programming language.

Diagram : class2.1

Programming language acts like a mediator or interface between user and computer.

Programming language divided into two types.

1) Low Level Language

2) High Level Language

1) Low Level Language

A language which is understand by a computer easily is called low level language.

A language which is computer dependent is called low level language.

ex:

Machine Language

Assembly Language

Machine Language

It is a fundamental language of a computer which is combination of 0's and 1's.

It is also known as binary language.

Our computer may understands many languages but to understand machine language it does not required any translator.

Advantages

> A program written in machine language consumes very less memory.

> It does not required any translator.

> It is more efficient when compare to other languages.

Disadvantages:

> It is a burden on a programmer to remember dozen's of binary code.

> If anywhere error raised in our program then locating and handling that error becomes difficult.

> Modifications can't be done easily.

Assembly Language

The second generation language came into an existence is called assembly language.

It is also known as symbolic language.

Assembly language can't understand by a computer directly. We required translator.

We have three translators.

i) Assembler

ii) Compiler

iii) Interpreter

i) Assembler

It is one of the translator which converts assembly code to machine code.

Merits:

> If anywhere error raised in our program then locating and handling that error becomes easy.
> Modifications can be done easily.

Demerits:

> It is a mind trick to remember all symbolic code.
> It requires translator.
> It is less efficient when compare to machine language.

Q) What is Debugging?

Bug is also known as error.
The process of eliminating the bugs from the application is called debugging.

2) High Level Language

A language which is understand by a user easily is called high level language.

A language which is user dependent is called high level language.

ex:

C++, Java, .NET, Python and etc.

High level language can't understand by a computer directly. We required translators.

compiler

It is used to compile and execute our program at a time.

interpreter

It is used to execute our program line by line procedure.

Advantages:

> It is easy to learn and easy to use because it is similar to English language.

> Debugging can be done easily.

> Modifications can be done easily.

Disadvantages:

> A program written in high level language consumes huge amount of memory.

> It requires translator.

> It is not efficient when compare to low level language.

Day:8 01-08-26 Saturday

==========

Flavours in Java

==========

We have three flavours in java.

Java
|----------|----------|
JSE/J2SE JEE/J2EE JME/J2ME
(Java Standard Edition) (Java Enterprise Edition) (Java Micro Edition)

> Standalone App > Distributed App > Mobile App

> Desktop App > Enterprise App

> Two-tier App > ERP App

> N-tier App

> Standalone App

A normal java program which contains main method is called standalone application.

ex:
```
class Test
{
public static void main(String[] args)
{

}
}
```

> Desktop App

It is a software application specially designed to perform particular task.
ex:
Control panel
Recycle Bin
VLC media player
and etc.

> Two-tier App

Having more then one tier is called two tier application.

Diagram: class3.1

Diagram: class3.2

> Distributed App

Diagram : class3.3

In client/server architecture if multiple clients sending the request to main server then main server will distribute those request to it's parallel servers to reduce the burden of main server such type of application is called distributed application.

> Enterprise App

An application which deals with large business complex logic with the help of middleware services is called enterprise application.

Here middleware services means authentication, authorization, malware production, security and etc.

ex:

Zoom Meeting

Skype

Online shopping website

and etc.

> ERP App

==========

ERP stands for Enterprise Resource Planning.

It is a centralized software system that helps a company to manage core processes or businesses including finance, human resources, and supply chain management.

Diagram: class3.4

> N-tier App

Having more then two tiers is called N-tier application.

Note:

JME flavour is outdated from java because to develop mobile applications we got many alternate technologies like Android, Flutter, React Native, Swift programming, kotlin and etc.

Interview Question

==========

Q) What is Java ?

Java is a versatile programming which is used to develop various applications like standalone applications, desktop applications, distributed applications, ERP applications, enterprise applications, mobile applications, networking applications and etc.

Day:9 03-08-26 Monday

==========

Naming conventions in java

==========

In java, uppercase letters consider as different and lowercase letters consider as different. Hence we consider java is a case sensitive programming language.

As java is a case sensitive, we must and should follow naming conventions for following things.

ex:

classes

interfaces

variables

methods

keywords

packages &

constants

classes

In java, a class name must and should starts with uppercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

Predefined classes Userdefined classes

---------- ----------

System Test

String TestApp

FileWriter DemoExample

BufferedReader MyProgram

and etc. and etc.

interfaces

In java, an interface name must and should starts with uppercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

Predefined interfaces Userdefined interfaces

---------- ----------

Serializable ITest

Cloneable IDemoApp

ListIterator IExampleApp

Enumeration IMyProgram

and etc. and etc.

variables

In java, a variable name must and should starts with lowercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

Predefined variables Userdefined variables

---------- ----------

out empId

length studName

err deptNo

in employeeSalary

and etc. and etc.

methods

In java, a method name must and should starts with lowercase letter and if it contains multiple words then each inner word must starts with initcap.

ex:

predefined methods userdefined methods

---

hashCode() getDetails()

toString() setInfo()

getName() calculateBill()

setPriority() depositMoney()

and etc. and etc.

keywords

In java, all keywords written in lowercase letters.

ex:

Predefined keywords

---

if

else

do

while

for

switch

case

public

static

void

and etc.

packages

In java, all packages are written in lowercase letters.

ex:

Predefined packages	Userdefined packages
java.lang	com.ihub.www
java.io	in.qualitythought.www
java.util	in.raisetech.www
java.util.stream	com.skilltransform.www
java.text	and etc.
java.time	
java.sql	
and etc.	

constants

In java, all constants are written in uppercase letters.

ex:

Predefined constants	Userdefined constants
MAX_PRIORITY	LIMIT
MIN_PRIORITY	N
MAX_PRIORITY	URL
MAX_VALUE	USERNAME
MIN_VALUE	PASSWORD
and etc.	and etc.

Assignment

==========

1) class : RaiseTech

2) interface : IRaiseTech

3) variable : raiseTech

4) method : raiseTech()

5) package : com.raisetech.www

6) constant : RAISE_TECH

Interview Questions

Q) What is package?

A package is a collection of classes and interfaces.

Q) Which package consider as default package in java?

java.lang package

Day:10_____ 04-08-26 Tuesday
==========

Java
====
Version : JDK 25 or JDK 26

Vendor : Oracle Corporation

Creator : James Gosling

Open source : Open source

Website : www.oracle.com/in/java/

Tutorials : www.geeksforgeeks.org
www.w3schools.com
www.tutorialspoint.com and etc.
Download link :
https://www.oracle.com/in/java/technologies/downloads/#jdk26-windows

Steps to setup environment variables
==========

step1:

Make sure JDK software installed successfully.

step2:

Copy "bin" directory from "JAVA" home folder.

ex:

C:\Program Files\Java\jdk-25\bin

step3:

Paste "bin" directory in environment variables.

ex:

Right click to This PC --> properties --> Advanced System settings --> Environment variables --> system variables --> click to new button --> variable name : path variable value : C:\Program Files\Java\jdk-25\bin; --> ok --> ok --> ok.

step4:

Check environment variables setup done perfectly or not.

ex:

cmd> javap

cmd> java -version

Steps to develop first java application

==========

step1:

Create a "javaprog" folder inside "D" drive.

step2:

Open the notepad and develop Hello World program.

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("Hello World");
}
}
```

step3:

Save java program with same name as class name inside javaprog folder.

step4:

Open the command prompt from "javaprog" location.

step5:

Compile java program by using below command.

ex:

javac Test.java

|

filename

step6:

Execute java program by using below command.

ex:

java Test

|

classname

Note:

Since Java 11, we can execute our program directly.

ex:

java Test.java

Day:11_____ 05-08-26 Wednesday

==========

History of Java

==========

In 1990, Sun Micro System took one project to develop a software called consumer electronic device which can be controlled by a remote like setup box. That time project was called Stealth project and later it is renamed to Green project.

James Gosling, Mike Sharadin and Patrick Naughton were there to develop the project and they have met in a place called Aspan/Colorado to start the work with graphic system. James Gosling thought to use C and C++ languages to develop the project. But the problem what they have faced is C and C++ languages are system dependent. Then James Gosling decided why don't we create our own programming language which is system independent.

In 1991, they have developed a programming language called an OAK. OAK is a national tree for many countries like Germany, France, USA and etc.

Later in 1995, they have renamed OAK to JAVA. Java is a island of an Indonesia where first coffee of seed was produced. During the development of project they were consuming lot of coffee's. Hence a symbol of java is a cup of coffee with saucer.

Interview Questions

==========

Q) Who is the creator of java?

James Gosling

Q) In which year java was developed?

In 1995

Q) Java originally known as __ ?

OAK

Q) Is java platform dependent or independent?

Java consider as platform independent at byte code level.

Hence java is also known as WORA (Write Once Run Anywhere).

Diagram: class6.1

Q) Is JVM platform dependent or independent?

JVM consider as platform dependent.

Q) What is difference between JDK, JRE and JVM ?

JDK

JDK stands for Java Development Kit.

JDK is a installable software which consist Java Runtime Environment (JRE), Java Virtual Machine (JVM), compiler (javac), interpreter (java), an archiver (.jar), document generator (javadoc) and other tools needed for java application development.

JRE

JRE stands for Java Runtime Environment.

JRE is a part of a JDK which provides very good environment to run java applications only.

JVM

JVM stands for Java Virtual Machine.

JVM is a part of a JRE which is used to execute our program line by line procedure and converts byte code to machine code.

Diagram: class6.2

Day:12_____ 06-08-26 Thursday

==========

Internal Architecture of JVM

==========

Diagram: class7.1

Java application contains java code instructions. Once if we compiled java code instructions convert to byte code instructions in .class file.

JVM invokes one module called class loader/subsystem to load all the byte code instructions from .class file. The work of class loader is to check these byte code instructions are proper or not. If they are not proper then it will refuse the execution. If they are proper then it allocates memory.

We have five memories in java.

1) Method Area

It contains code of a class, code of a variable and code of a method.

2) Heap

Our object creation stores in heap area.

3) Java Stack

Java methods store in method area. But to execute those method we required some memory and that memory is allocated in java stack.

4) PC register

It is a program counter register which is used to track the address of instructions.

5) Native Method Stack

Java methods execute in method area.

Similarly native methods execute in native method stack.

To execute native methods we required one program called Native method interface.

Execution Engine

Execution Engine contains interpreter and JIT compiler.

Whenever JVM loads byte code instructions from .class file. It simultaneously uses interpreter

and JIT compiler.

Interpreter is used to execute our program line by line procedure.

JIT compiler is used to increase the execution speed of our program.

Finally, JVM converts byte code to machine code.

Interview Questions
==========
Q) How many memories are there in java?

We have five memories in java.
1) Method Area
2) Heap
3) Java Stack
4) PC Register
5) Native Method Stack

Q) What is JIT compiler?

JIT compiler is a part of a JVM which is used to increase the execution speed of our program.

Q) What is Native method in java?

A method which is developed by using some other language is called native method.
To execute native methods we required one program called Native method interface.
All native methods store in native method stack.

Features of Java
==========
We have following important features of java.

1) Simple
2) Object oriented
3) Platform independent
4) Portable

5) Architecture Neutral
6) Robust
7) High Performance
8) Highly secured
9) Multithreaded
10) Dynamic
11) Distributed
and etc.

Day:13_____ 07-08-26 Friday and Day:14_____ 10-08-26 continues....
==========

Identifiers
==========
A name given to a program elements is called identifier.

Program elements are variables , methods, classes , interfaces and etc.

ex:

```
class Test
{
public static void main(String[] args)
{
int x = 10;

System.out.println(x)
}
}
```

Here Test, main, String, args, x , System, out, println are identifiers.

Rules to declare an identifiers

Rule1:

Identifier will accept following characters.
ex:

A-Z
a-z
0-9
_
$

Rule2:

If we take other characters then we will get compile time error.
ex:
int empId;
int emp_id;
int emp$alary;
int dept@no; //invalid

Rule3:

Every identifier must and should starts with alphabet, underscore or
dollar symbol but not with digit.
ex:
int a1234;
int _abcd;
int $alary;
int 1abcd; //invalid

Rule4:

We can't take reserved words as an identifier name.
ex:
int if; //invalid
int else; //invalid
int for; //invalid
int do; //invalid

Rule5:

Every identifier is a case sensitive.
ex:

int number;
int Number;

Rule6:

There is no length limit for an identifier but it is not
recommended to take more then 15 characters.

Datatypes
==========
Datatype describes what type of value we want to store inside a variable.

Datatype also tells how much memory has to be created for a variable.

In java, Datatypes are divided into two types.

1) Primitive Datatypes

2) Non-Primitive Datatypes

Diagram: class8.1

byte

It is a smallest datatype in java.

Size : 1 byte (8 bits)

Range : -128 to 127 (-2^7 to 2^7-1)

ex:
1) byte b = 10;
System.out.println(b); // 10

2) byte b = 130;
System.out.println(b); // C.T.E

3) byte b = 10.56;

System.out.println(b); // C.T.E

short

It is rarely used datatype in java.

Size : 2 bytes (16 bits)

Range : -32768 to 32767 (-2^15 to 2^15-1)

ex:
1) byte b = 10;
short s = b;
System.out.println(s); // 10

2) short s = 10.56;
System.out.println(s); // C.T.E

3) short s = "Hi";
System.out.println(s); // C.T.E

----------continues----------

int

It is mostly used datatype in java.

Size : 4 bytes (32 bits)

Range : -2147483648 to 2147483647 (-2^31 to 2^31-1)

ex:
1) int i = 10.56;
System.out.println(i); // C.T.E

2) int i = "true";
System.out.println(i); // C.T.E

3) int i = 'a';
System.out.println(i); // 97

Note:

In java, for every character we have universal unicode value.
ex:
a - 97
A - 65

long

If int datatype is not enough to hold large value then we need to use long datatype.

Size : 8 bytes (64 bits)

Range : (-2^63 to 2^63-1)

ex:
1) long l = "hi";
System.out.println(l); // C.T.E

2) long l = 10.56;
System.out.println(l); // C.T.E

3) long l = true;
System.out.println(l); // C.T.E

4) long l = 'A';
System.out.println(l); // 65

float double
------- ---------
If we need 4 to 6 decimal point of accuracy If we need 14 to 16 decimal point of
then we need to use float. accuracy then we need to use double.

Size : 4 bytes (32 bits) Size : 8 bytes (64 bits)

Range : -3.4e38 to 3.4e38 Range: -1.7e308 to 1.7e308

To declare float value we need to To declare double value we need to
suffix with 'f' or 'F'. suffix with 'd' or 'D'.
ex: ex:
10.56f 10.56d

ex:
---
1) float f = 10;
System.out.println(f); // 10.0

2) float f = 10.56f;
System.out.println(f); // 10.56

3) float f = 'a';
System.out.println(f); // 97.0

4) float f = "hi";
System.out.println(f); // C.T.E

5) float f = true;
System.out.println(f); // C.T.E

ex:
---
1) double d = 10;
System.out.println(d); // 10.0

2) double d = 10.56d;
System.out.println(d); // 10.56

3) double d = 'a';
System.out.println(d); // 97.0

4) double d = "hi";
System.out.println(d); // C.T.E

5) double d = true;
System.out.println(d); // C.T.E

boolean

It is used to represent boolean values either true or false.

Size : (Not Applicable)
Range : (Not Applicable)

ex:
1) boolean b = "true";
System.out.println(b); // C.T.E

2) boolean b = TRUE;
System.out.println(b); // C.T.E

3) boolean b = false;
System.out.println(b); // false

char

It is a single character which is enclosed in a single quotation.

Size : 2 bytes (16 bits)

Range : 0 to 65535

ex:
1) char ch = 'a';
System.out.println(ch); // a

2) char ch = 65;
System.out.println(ch); // A

3) char ch = "B";
System.out.println(ch); // C.T.E

4) char ch = 'ab';
System.out.println(ch); // C.T.E

Diagram: class8.2

Day:15_____ 11-08-26 Tuesday and Day:16_____ 12-08-26 Wednesday continues....
==========

Q) Write a java program to display size of int datatype?

```
class Test
{
void main()
{
System.out.println(Integer.BYTES);
}
}
```

Q) Write a java program to display size of long datatype?

```
class Test
{
void main()
{
System.out.println(Long.BYTES);
}
}
```

Q) Write a java program to display range of short datatype?

```
class Test
{
void main()
{
System.out.println(Short.MIN_VALUE);
System.out.println(Short.MAX_VALUE);
```

```
}
}
```

Q) Write a java program to display range of int datatype?

```
class Test
{
void main()
{
System.out.println(Integer.MIN_VALUE);
System.out.println(Integer.MAX_VALUE);
}
}
```

Q) Is java purely object oriented or not?

No, java will not consider as purely object oriented programming language because it does not support many OOPS concepts like multiple inheritance, operator overloading and more ever we depends upon primitive datatypes which are non-objects.

Types of variables

==========

A name given to a memory location is called variable.

Purpose of variable is used to store the data.

In java, variables are divided into two types.

1) Primitive variables

It is used to represent primitive values.

2) Reference variables

It is used to represent object reference.

ex:

```
Student s = new Student();
|
```

reference variable

Based on the position and execution these variables are divided into three types.

1) Instance variables / Non-static variables

2) Static variables / Global variables

3) Local variables / Temporary variables

1) Instance variables

A value of a variable which is varied from object to object is called instance variable.

Instance variable allocates memory at the time of object creation and it will destroy at the time of object destruction. Hence scope of instance variable is same as scope of an object.

Instance variables store in heap area as a part of an object.

Instance variable must and should declare immediately after the class but not inside methods, blocks and constructors.

Instance variable we can access directly from instance area but we can't access directly from static area.

To access instance variable from static area we need to create object reference.

ex:1

```
class Test
{
//instance variable
int i = 10;

public static void main(String[] args)
{
System.out.println(i);// C.T.E
}
}
```

ex:2

```
class Test
{
//instance variable
int i = 10;

public static void main(String[] args)
{
Test t = new Test();
System.out.println(t.i); // 10
}
}
```

Note:

If we won't initialize any value to instance variable then JVM will initialized default values.

ex:3

```
class Test
{
//instance variable
boolean b;

public static void main(String[] args)
{
Test t = new Test();
System.out.println(t.b); // false
}
}
```

In java, for every object a seperate copy of instance variable will be created.

ex:4

```
class Test
{
//instance variable
int i = 10;

public static void main(String[] args)
{
Test t1 = new Test();
Test t2 = new Test();
System.out.println(t1.i); // 10
System.out.println(t2.i); // 10
t1.i=100;
System.out.println(t1.i); // 100
System.out.println(t2.i); // 10
}
}
```

ex:5

```
class Test
{
public static void main(String[] args)
{
Test t = new Test();
t.methodOne();
}
//non-static method
public void methodOne()
{
System.out.println("instance-method");
}
}
```

Assignment

==========

Q) Jack and John both are best friends in a town. One day while going to school they saw one beggar. Both have decided to help that poor guy. Jack gave 100 rupees from his school fee and John gave 50 rupees from his pocket money. Write a java program to find out total contribution they have done to assist a needy person.

```
class Test
{
//instance variable
int a=100,b=50;

public static void main(String[] args)
{
Test t = new Test();
t.sum();
}
//instance method
public void sum()
{
int c = a + b;
System.out.println(c);
}
}
```

----------continues----------

2) Static variables

==========

A value of a variable which is not varied from object to object is called static variable.

Static variable allocates memory at the time of class loading and it will destroy at the time of class unloading. Hence scope of static variable is same as scope of .class file.

Static variables store in method area.

Static variables must and should declare immediately after the class using static keyword but not inside methods, blocks and constructors.

Static variable we can access directly from instance area as well as from static area.

Static variable we can access by using object reference and class name.

ex:1

```
class Test
{
//static variable
static int i = 10;

public static void main(String[] args)
{
System.out.println(i); // 10

Test t = new Test();
System.out.println(t.i); // 10

System.out.println(Test.i); // 10
}
}
```

Note:

If we won't initialize any value to static variable then JVM will initialized default values.

ex:2

```
class Test
{
//static variable
static String s;

public static void main(String[] args)
{
System.out.println(s); // null

Test t = new Test();
System.out.println(t.s); // null

System.out.println(Test.s); // null
}
```

```
}
```

In java, only one copy of static variable will be created and shared to multiple objects.

ex:3

```
class Test
{
//static variable
static int i = 10;

public static void main(String[] args)
{
Test t1 = new Test();
Test t2 = new Test();
System.out.println(t1.i); // 10
System.out.println(t2.i); // 10
t1.i=100;
System.out.println(t1.i); // 100
System.out.println(t2.i); // 100
}
}
```

ex:4

```
class Test
{
public static void main(String[] args)
{
methodOne();

Test t = new Test();
t.methodOne();

Test.methodOne();
}
//static method
public static void methodOne()
```

```
{
System.out.println("static-method");
}
}
```

3) Local variables

==========

To meet temporary requirements, a programmer will declare some variables inside methods, blocks and constructors such type of variables are called local variables.

Local variable allocates memory at the time of execution block and it will destroy when execution block is executed. Hence scope of local variable is same as scope of execution block where it is declared.

Local variables store in java stack memory.

ex:1

```
class Test
{
public static void main(String[] args)
{
//local variable
int i = 10;
System.out.println(i); // 10
}
}
```

Note:

If we won't initialize any value to local variable then JVM will not initialized default values.

ex:2

```
class Test
{
```

```
public static void main(String[] args)
{
//local variable
int i;
System.out.println(i);
}
}
// C.T.E variable i might not have been initialized
```

A local variable will accept only one modifier i.e final.

ex:3

```
class Test
{
public static void main(String[] args)
{
//local variable
final int i = 10;
System.out.println(i);
}
}
```

var keyword

==========

A var keyword introduced in Java 10.

It is used to declare Local Variable Type Inference(LVTI).

ex:

```
var i = 10;

var i = 10.5d;

var i = "Hi";

var i = 'a';

var i = true;
```

Day:17_____ 13-08-26 Thursday

==========

Reserved Words

==========

Reserved words are predefined words which are reserved in java to associate some functionality or meaning. Such type of words are called reserved words.

Java supports 70 reserved words and it is classified into two types.

All reserved words written in lowercase.

Diagram: class10.1

Used keywords with respect to flow control

---

if

else

switch

case

default

for

while

do

break

continue

return

Used keywords with respect to Access modifiers

---

public

private

protected

Used keywords with respect to classes and interfaces

---

class
interface
enum
extends
implements
abstract
final
static
strictfp
sealed
permits

Used keywords with respect to exception handling

---

try
catch
finally
throw
throws
assert

Used keywords with respect to variables and methods

---

void
var
new
this
super
volatile
transient
synchronized

Used keywords with respect to datatypes.

---

int
long
short

byte
float
double
char
boolean

Used keywords with respect to package
---
package
import
module (Java 9+)

Used keywords with respect to object and reference handling
---
instanceof
typeof (reserved but unused in Java)
yield (Java 14+ switch expression)
record (Java 16+)
native

Used keywords with respect to misc reserved words
---
requires (Java 9 modules)
exports (Java 9 modules)
opens (Java 9 modules)
uses (Java 9 modules)
provides (Java 9 modules)
with (Java 9 modules)
to (Java 9 modules)
non-sealed (Java 17+)

var-arg method
==========
Untill 1.4 version , it is not possible to declare a method with variable number of arguments. But from 1.5 version onwards it is possible to declare a method with variable number of arguments.

We can declare var-arg method as follow.

ex:

var-arg parameter

|

---

void methodOne(int... i)

|

ellipse

Here var-arg parameter is a replacement of single dimensional array.

ex:

... ---> []

We can invoke var-arg method with any number integer values including zero.

ex:1

```
class Test
{
public static void main(String[] args)
{
methodOne();
}
public static void methodOne()
{
System.out.println("0-arg method");
}
}
```

ex:2

```
class Test
{
public static void main(String[] args)
{
methodOne(10);
}
```

```
public static void methodOne(int i)
{
System.out.println("one-arg method");
}
}
```

ex:3

```
class Test
{
public static void main(String[] args)
{
methodOne(10,20);
}
public static void methodOne(int i,int j)
{
System.out.println("two-arg method");
}
}
```

ex:4

```
class Test
{
public static void main(String[] args)
{
methodOne();
methodOne(10);
methodOne(10,20);
methodOne(10,20,30);
}
public static void methodOne(int... i)
{
System.out.println("var-arg method");
}
}
```

In realtime we will use var-arg method when number of inputs are not fixed to a method.

ex:

```
class Zomato
{
public static void main(String[] arsg)
{
//bookOrder("Burgur");
bookOrder("Pizza","Coke");
}
public static void bookOrder(String... items)
{
//System.out.println(items[0]);
System.out.println(items[0]+" "+items[1]);
}
}
```

Rule1:

We can declare var-arg parameter with general parameters.

ex:

```
void methodOne(int x,int... y)
```

Rule2:

If we mix var-arg parameter with general parameter then var-arg parameter must be last parameter.

ex:

```
void methodOne(int... x,int y) //invalid
void methodOne(int x,int... y) //valid
```

Rule3:

A var-arg method can have only one var-arg parameter.

ex:

```
void methodOne(int... x) //valid
void methodOne(int... x,int... y) //invalid
```

Java Source File Structure

==========

case1:

A java program can have mutiple classes.

case2:

If a java program contains multiple classes then we need to check
which class contains main method and that class treated as main class.

ex:

Student.java

```
class Recordings
{
String src="Good Drive";
}
class Student
{
public static void main(String[] args)
{
Recordings r = new Recordings();
System.out.println(r.src);
}
}
```

If we compile above program we will get two .class files i.e
Recordings.class and Student.class.

case3:

If a java program contains multiple classes with main method then we need to declare
atleast one class as public and that public class treated as main class.

ex:

A.java

```
public class A
{
public static void main(String[] args)
```

```
{
System.out.println("A-class");
}
}
class B
{
public static void main(String[] args)
{
System.out.println("B-class");
}
}
```

If we compile above program we will get two .class files i.e

A.class and B.class.

ex:

javac A.java

We can execute above program as follow.

ex:

java A

java B

Day:18_____ 14-08-26 Friday

==========

Main method

==========

Our program contains main method or not. Either it is properly declared or not. It is not a responsibility of a compiler to check. It is a duty of a JVM to check for main method always at runtime. If JVM won't find main method then it will throw one runtime error called main method not found.

JVM always looks for main method with following signature.

signature:

---

public static void main(String[] args)

Before Java 21, We can perform following changes in main method.

1) Order of modifiers is not important incase of public static we can declare static public also.
ex:
static public void main(String[] args)

2) We can declare String[] in following acceptable formats.
ex:
public static void main(String[] args)
public static void main(String []args)
public static void main(String args[])

3) We can replace args with any java valid identifier.
ex:
public static void main(String[] raisetech)

4) We can change String[] with var-arg parameter.
ex:
public static void main(String... args)

5) Main method will accept following modifiers.
ex
synchronized
strictfp (deprecated from Java 17)
final

Since Java 21 , we can perform following changes in main method.

1) public modifier is optional.
ex:
static void main(String[] args)

2) We can declare instance main method.
ex:
void main(String[] args)

3) String[] is optional to declare.
ex:
void main()

Q) Explain main method in java?

public

JVM wants to call main method from anywhere.
static

JVM wants to call main method without using object reference.
void

Main method does not return anything to JVM.
main

It is a identifier given to a main method.

String[] args

It is a command line arguments.

Command Line Arguments
==========
Arguments which are passing through command prompt such type of arguments are called command line arguments.
ex:
cmd> java -version
gitbash> git status
$> mkdir myfolder

In command line arguments we need to pass our input values at runtime command.

ex:
javac Test.java

java Test 101 Alan M 1000.0
| | | |_____ args[3]
| | |_____ args[2]
| |_____ args[1]
|_____ args[0]

ex:
---
class Test
{
void main(String[] args)
{
System.out.println(args[0]);
System.out.println(args[1]);
System.out.println(args[2]);
System.out.println(args[3]);
}
}

Here args.length returns size of command line arguments.

ex:
---
class Test
{
void main(String[] args)
{
for(int i=0;i<args.length;i++)
{
System.out.println(args[i]);
}
}
}

Day:19_____ 17-08-26 Monday
==========
========

System.out.println()

==========

It is a output statement in java.

Whenever we want to display any userdefined statements , data or expression then we need to use output statement in java.

syntax:

---

static variable

|

System.out.println()

| |

predefined predefined method

final class

Diagram: class12.1

ex:

```
class Test
{
void main()
{
System.out.print("RAISE");
System.out.print("TECH");
}
}
```

o/p:

RAISETECH

ex:

```
class Test
{
void main()
{
System.out.println("RAISE");
```

```
System.out.println("TECH");
}
}
```

o/p:

RAISE

TECH

ex:

```
class Test
{
void main()
{
double d = 10.5478965d;
System.out.printf("%.2f",d);
}
}
```

Various ways to display the output in java

---

1) custom message

ex:

```
System.out.println("Hello World");
```

2) int i = 10

ex:

```
System.out.println(i);
System.out.println("The value is = "+i);
System.out.println(i+" is the value");
```

3) int i=10,j=20;

ex:

```
System.out.println(i+" "+j);
System.out.println(i+" and "+j);
```

4) int i=1,j=2,k=3;

ex:

```
System.out.println(i+" "+j+" "+k);
```

Q) What is the difference between System.out.println() and System.err.println()?

System.out.println()

It is used to display standard output.

It display the output only on console.

System.err.println()

It is used to display error output.

It display the output on console as well as redirect to physical file.

ex:

```
class Test
{
void main()
{
System.err.println("Welcome to Java Batch");
}
}
```

o/p:

```
javac Test.java
java Test 2>abc.txt
```

Q) Write a java program to display Hello World without using print stmt?

```
class Test
{
void main()throws java.io.IOException
{
String str = "Hello World";
System.out.write(str.getBytes());
}
}
```

Day:20_____ 18-08-26 Tuesday

==========

Fully Qualified Name

==========

Fully qualified name means we will declare a class or interface along with package name.

ex:

java.io.File(C)

java.util.Iterator(I)

Fully qualified name improves readability of our code.

The main purpose of fully qualified name is to avoid import confict.

ex:

```
import java.util.Date;
import java.sql.Date;
```

ex:

```
class Test
{
void main()
{
java.util.Date d = new java.util.Date();
System.out.println(d);
}
}
```

Import statement

==========

A import keyword in java makes other classes and interfaces into visibility in current class.

Whenever we use import statement we should not use fully qualified name.

In java, we have three import statements.

1) Explicit class import

2) Implicit class import

3) Static import

1) Explicit class import

This type of import statement is highly recommended to use because it improves readability of our code.

ex:

```
import java.util.Date;
class Test
{
void main()
{
Date d = new Date();
System.out.println(d);
}
}
```

ex:

```
import java.time.LocalDate;
import java.time.LocalTime;
class Test
{
void main()
{
LocalDate date = LocalDate.now();
System.out.println(date);

LocalTime time = LocalTime.now();
System.out.println(time);
}
}
```

2) Implicit class import

This type of import statement is not recommended to use because it reduce readability of our code.

ex:

--

```
import java.time.*;
class Test
{
void main()
{
LocalDate date = LocalDate.now();
System.out.println(date);

LocalTime time = LocalTime.now();
System.out.println(time);
}
}
```

3) Static import

It is introduced in java 1.5 version.

Using static import we can call static members(static variables and static methods) directly.

Often use of static import makes our program complex and unreadable.

ex:

```
import static java.lang.System.*;
class Test
{
void main()
{
out.println("stmt1");
out.println("stmt2");
out.println("stmt3");
}
```

}

ex:

```
import static java.lang.System.*;
class Test
{
void main()
{
out.println("stmt1");
exit(0);
out.println("stmt3");
}
}
```

Day:21_____ 19-08-26 Wednesday

==========

EditPlus

==========

Download link : https://www.editplus.com/download.html

Java Basic Programs

==========

1) Write a java program to perform sum of two numbers?

```
import java.util.Scanner;
class Example1
{
void main()
{
Scanner sc = new Scanner(System.in);

System.out.println("Enter the first number : ");
int a = sc.nextInt();//10
```

```
System.out.println("Enter the second number : ");
int b = sc.nextInt();//20

int c = a + b;

System.out.println("sum of two numbers is ="+c);
}
}
```

2) Write a java program to find out square of a given number?

```
import java.util.Scanner;
class Example2
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt(); // 5

int square = n * n;

System.out.println("Square of a given number is = "+square);
}
}
```

3) Write a java program to find out cube of a given number?

```
import java.util.Scanner;
class Example3
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt(); // 5
```

```
int cube = (int)Math.pow(n,3);
System.out.println("cube of a given number is ="+cube);
}
}
```

4) Write a java program to find out greatest of two numbers?

```
import java.util.Scanner;
class Example4
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt(); // 20
System.out.println("Enter the second number : ");
int b = sc.nextInt(); // 30

int max = Math.max(a,b);
System.out.println(max+" is greatest");
}
}
```

5) Write a java program to find out greatest of three numbers?

```
import java.util.Scanner;
class Example5
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt(); // 20
System.out.println("Enter the second number : ");
int b = sc.nextInt(); // 30
System.out.println("Enter the third number : ");
int c = sc.nextInt(); // 5
```

```
int max = Math.max(Math.max(a,b),c);
System.out.println(max+" is greatest");
}
}
```

6) Write a java program to find out area of a circle?

```
import java.util.Scanner;
class Example6
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the radius : ");
int r = sc.nextInt(); // 5

double area = 3.14 * Math.pow(r,2);

System.out.println("Area of a circle is = "+area);
}
}
```

7) Write a java program to find out perimeter of a circle?

```
import java.util.Scanner;
class Example7
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the radius : ");
int r = sc.nextInt(); // 5

double perimeter = 2 * Math.PI * r;

System.out.printf("Perimeter of a circle is = %.2f",perimeter);
}
```

```
}
```

8) Write a java program to perform swapping of two numbers?

```
import java.util.Scanner;
class Example8
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt(); // 10
System.out.println("Enter the second number : ");
int b = sc.nextInt(); // 20

System.out.println("Before swapping a="+a+" and b="+b);

int temp = a;
a = b;
b = temp;

System.out.println("After swapping a="+a+" and b="+b);
}
}
```

9) Write a java program to display swapping of two numbers without using third variable?

```
import java.util.Scanner;
class Example9
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt(); // 10
System.out.println("Enter the second number : ");
int b = sc.nextInt(); // 20
```

```
System.out.println("Before swapping a="+a+" and b="+b);

a = a + b;
b = a - b;
a = a - b;

System.out.println("After swapping a="+a+" and b="+b);
}
}
```

10) Write a java program to accept dob and find out person's age?

```
import java.time.LocalDate;
import java.time.Period;
class Example10
{
void main()
{
LocalDate dob = LocalDate.of(2005,4,17);
LocalDate curr_date = LocalDate.now();
Period p = Period.between(dob,curr_date);
int age = p.getYears();
System.out.println("Your age is ="+age+" "+months);
}
}
```

11) Write a java program to convert time to seconds?

Input:
Hours : 10
Minutes : 30
Seconds : 23

Output:
37823

```
class Example11
```

```
{
void main()
{
int hours = 10;
int minutes = 30;
int seconds = 23;

int totalSeconds = (hours*3600)+(minutes*60)+seconds;

System.out.println(totalSeconds);
}
}
```

12) Write a java program to accept one salary and find out 10% of TDS?

```
import java.util.Scanner;
class Example12
{
void main()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the salary : ");
int salary = sc.nextInt();

double tds = (double)salary*10/100;

System.out.println("10% of TDS is = "+tds);
}
}
```

Day:22_____ 20-08-26 Thursday

==========

Typecasting

==========

The process of converting from one datatype to another datatype is called typecasting.

In java , typecasting can be performed in two ways.

1) Implicit typecasting

2) Explicit typecasting

1) Implicit typecasting

If we want to store smaller value into a bigger variable then we need to use implicit typecasting.

A compiler is responsible to perform implicit typecasting.

There is no possibility to loss the information.

It is also known as widening or upcasting.

We can perform implicit typecasting as follow.

syntax:

---

byte --> short

-->

int --> long --> float --> double

-->

char

ex:

```
class Test
{
public static void main(String[] args)
{
byte b = 10;
int i = b;
System.out.println(i);// 10
}
```

```
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
char ch = 'a';
long l = ch;
System.out.println(l);// 97
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;
float f = i;
System.out.println(f); // 10.0
}
}
```

2) Explicit typecasting

If we want to store bigger value into a smaller variable then we need to use explicit typecasting.

A programmer is responsible to perform explicit typecasting.

There is a possibility to loss the information.

It is also known as narrowing or downcasting.

We can perform explicit typecasting as follow.

ex:

byte <-- short

<--

int <-- long <-- float <-- double

<--

char

ex:

```
class Test
{
public static void main(String[] args)
{
double d = 10.56d;
int i = (int)d;
System.out.println(i); // 10
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 65;
char ch = (char)i;
System.out.println(ch); // A
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 130;
```

```
byte b = (byte)i;
System.out.println(b); // -126
}
}
```

Types of blocks

==========

A block is a set of statements which is enclosed in a curly braces i.e {}.

In java we have three types of blocks.

1) Instance block

2) Static block

3) Local block

1) Instance block

Instance block is used to initialize instance variables.

Instance block must and should declare immediately after the class but not inside methods and constructors.

Instance block is executed automatically when we create object.

We can declare instance block as follow.

syntax:

```
//instance block
{
-
- // set of statements
-
}
```

ex:

```
class Test
{
//instance block
{
System.out.println("instance-block");
}
public static void main(String[] args)
{
System.out.println("main-method");
}
}
```

o/p:

main-method

ex:

```
class Test
{
//instance block
{
System.out.println("instance-block");
}
public static void main(String[] args)
{
System.out.println("main-method");
Test t = new Test();
}
}
```

o/p:

main-method

instance-block

ex:

```
class Test
```

```
{
//instance block
{
System.out.println("instance-block");
}
public static void main(String[] args)
{
Test t1 = new Test();
System.out.println("main-method");
Test t2 = new Test();
}
}
```

o/p:

instance-block

main-method

instance-block

ex:

--

```
class Test
{
//instance variable
int i;

//instance block
{
i = 100;
}

public static void main(String[] args)
{
Test t = new Test();
System.out.println(t.i); // 100
}
}
```

2) Static block

Static block is used to initialize static variables.

Static block must and should declare immediately after the class but not inside methods and constructors.

Static block will execute automatically when class is loaded.

We can declare static block as follow.

syntax:

```
// static block
static
{
-
- //set of statements
-
}
```

ex:

```
class Test
{
//static block
static
{
System.out.println("static-block");
}

public static void main(String[] args)
{
System.out.println("main-method");
}
}
```

o/p:

```
static-block
main-method
```

ex:

```
class Test
{
//static variables
static int i;

//static block
static
{
i = 300;
}

public static void main(String[] args)
{
System.out.println(i); // 300
}
}
```

3) Local block

Local block is used to initialize local variables.

Local block must and should declare inside methods and constructors.

local block will execute just like a normal statement.

We can declare local block as follow.

syntax:

```
// syntax
{
-
- //set of statements
-
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");

//local block
{
System.out.println("stmt2");
}

System.out.println("stmt3");
}
}
```

o/p:

stmt1

stmt2

stmt3

ex:

```
class Test
{
public static void main(String[] args)
{
//local variable
int i;

//local block
{
i = 300;
}

System.out.println(i);
}
```

}

Day:23_____ 21-08-26 Friday

==========

Operators

==========

Operator is a symbol which is used to perform some operations on operands.

ex:

c = a + b;

Here

a,b and c are operands

= and + are operators

We have following list of operators in java.

1) Arithmetic operators
2) Assignment operators
3) Ternary/Conditional operators
4) Logical operators
5) Bitwise operators
6) Relational operators
7) Shift operators
8) Unary operators

1) Arithmetic operators

Operators Precedence

---

1) *, /, %
2) +, -
3) <<,>>

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 3+6/4+7%3+6*2+9/10+3%30+6-20;
System.out.println(i);
}
}
/*
3+6/4+7%3+6*2+9/10+3%30+6-20
3+6/4+7%3+12+9/10+3%30+6-20
3+1+7%3+12+0+3%30+6-20
3+1+1+12+0+3+6-20
26-20
6
*/
```

2) Assignment operators

---

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i = 20;

i = 30;

System.out.println(i); // 30
}
}
```

Note:

Reinitialization is possible in java.

ex:

```
class Test
{
public static void main(String[] args)
{
final int i = 10;

i = 20;

i = 30;

System.out.println(i); // C.T.E
}
}
```

Note:

Final variables we can't modify or change.

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 1,2,3,4,5;
System.out.println(i); // C.T.E
}
}
```

ex:

--

```
class Test
{
public static void main(String[] args)
{
int i,j;
```

```
i = j = 10;

System.out.println(i+" "+j); // 10 10
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i+=5; // i = i + 5

System.out.println(i);//15
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i-=5; // i = i - 5

System.out.println(i);//5
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i*=5; // i = i * 5

System.out.println(i);//50
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i/=5; // i = i / 5

System.out.println(i);//2
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i%=5; // i = i % 5

System.out.println(i);//0
}
```

}

3) Ternary/Conditional operators

---

syntax:

(condition)?value1:value2;

ex:

--

```
class Test
{
public static void main(String[] args)
{
String s = (true)?"Hi":"Bye";
System.out.println(s);
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
boolean b = (5>20)?true:false;
System.out.println(b); // false
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
char ch = (5>2)?'T':'F';
```

```
System.out.println(ch); // T
}
}
```

Q) Write a java program to find out greatest of two numbers?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt(); // 10
System.out.println("Enter the second number : ");
int b = sc.nextInt(); // 20

int max = (a>b)?a:b;

System.out.println(max+" is greatest ");

}
}
```

Q) Write a java program to find out greatest three numbers?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt(); // 10
System.out.println("Enter the second number : ");
int b = sc.nextInt(); // 20
```

```
System.out.println("Enter the third number : ");
int c = sc.nextInt(); // 30

int max = (a>b)?((a>c)?a:c):((b>c)?b:c);

System.out.println(max+" is greatest ");

}
}
```

Day:24_____ 22-08-26 Saturday

==========

How to convert decimal to binary number

---

ex:

10 - decimal number

1010 - binary number

2|10

---- 0

2|5

---- 1

2|2 ^

---- 0 |

1 |

1010

How to convert binary number to decimal number

1010 - binary number

10 - decimal number

1010 <----

0*1 + 1*2 + 0*4 + 1*8

0 + 2 + 0 + 8
10

4) Bitwise operators
==========

Bitwise AND operator (&)

Bitwise AND operator deals with binary numbers.

Truth table

T T = T
T F = F
F T = F
F F = F

ex:

```
class Test
{
public static void main(String[] args)
{
int a=15,b=10;
int c = a & b;
System.out.println(c); //10
}
}
/*
15 - 1111
10 - 1010
---
& - 1010 <-----
```

0*1 + 1*2 + 0*4 + 1*8

0 + 2 + 0 + 8 = 10

*/

Bitwise OR operator (|)

Bitwise OR operator deals with binary numbers.

Truth table

T T = T
T F = T
F T = T
F F = F

ex:

```
class Test
{
public static void main(String[] args)
{
int a=15,b=10;
int c = a | b;
System.out.println(c); //15
}
}
/*
15 - 1111
10 - 1010
---
| - 1111 <-----

1*1 + 1*2 + 1*4 + 1*8

1 + 2 + 4 + 8 = 15
*/
```

Bitwise XOR operator (^)

Bitwise XOR operator deals with binary numbers.

Truth table

T T = F

T F = T

F T = T

F F = F

ex:

```
class Test
{
public static void main(String[] args)
{
int a=15,b=10;
int c = a ^ b;
System.out.println(c); //5
}
}
/*
15 - 1111
10 - 1010
---------
^ - 0101 <----

1*1 + 0*2 + 1*4 + 0*8
1 + 0 + 4 + 0
5
*/
```

Bitwise NOT operator (~)

---

syntax:

-(n+1)

ex:

```
class Test
{
public static void main(String[] args)
{
int n = ~10;
System.out.println(n); // -11
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int n = ~(-23);
System.out.println(n); // 22
}
}
```

5) Logical operators

==========

Logical AND operator (&&)

Logical AND operator deals with boolean values either true or false.

Truth table

T T = T

T F = F

F T = F

F F = F

ex:

```
class Test
```

```
{
public static void main(String[] args)
{
System.out.println(true && true);
System.out.println(true && false);
System.out.println(false && true);
System.out.println(false && false);
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
boolean b = (5>2) && (6<10);
System.out.println(b); // true
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
boolean b = (5>2) && (6<1);
System.out.println(b); // false
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
```

```
boolean b = (5>20) && (6<1);
System.out.println(b); // false
}
}
```

Logical OR operator (||)

Logical OR operator deals with boolean values either true or false.

Truth table

T T = T

T F = T

F T = T

F F = F

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println(true || true);
System.out.println(true || false);
System.out.println(false || true);
System.out.println(false || false);
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
boolean b = (5>20) || (6<1);
System.out.println(b); // false
```

```
}
}
```

ex:

```
---
class Test
{
public static void main(String[] args)
{
boolean b = (5>20) || (6<10);
System.out.println(b); // true
}
}
```

ex:

```
---
class Test
{
public static void main(String[] args)
{
boolean b = (5>2) || (6<10);
System.out.println(b); // true
}
}
```

Logical NOT operator (!)

---

logical NOT operators deals with boolean values.

ex:

```
--
class Test
{
public static void main(String[] args)
{
boolean b = !(5>2);
System.out.println(b); //false
}
```

}

ex:

```
class Test
{
public static void main(String[] args)
{
boolean b = !(5>20);
System.out.println(b); //true
}
}
```

Q) Write a java program to check given number is even or odd?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();
System.out.println(((n&1)==0)?"It is even":"It is odd");
}
}
/*
10 - 1010
1 - 0001
---------
& - 0000
*/
```

6) Relational operators
==========

```
class Test
{
public static void main(String[] args)
{
System.out.println(10 > 2); // true
System.out.println(10 < 20); // true
System.out.println(10 >= 2); // true
System.out.println(10 <= 10); // true
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println(10 == 10); // true
System.out.println(10 == 20); // false
System.out.println(10 != 20); // true
System.out.println(10 != 10); // false
}
}
```

7) Shift operators

==========

Right shift operator (>>)

---

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10 >> 2;
System.out.println(i); // 10/4 -> 2
```

```
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 100 >> 4;
System.out.println(i); // 100/16 = 6
}
}
```

Left Shift operator (<<)

---

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10 << 2;
System.out.println(i); // 10 * 4 = 40
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 100 << 3;
System.out.println(i); // 100 * 8 = 800
}
}
```

Assignment

==========

Q) What is unsigned right shift operator (>>>)?

Day:25_____ 24-08-26 Monday

==========

8) Unary operators

==========

Increment/Decrement (++/--)

We have two types of increment operators.

1) Post Increment

---

ex:

i++

2) Pre Increment

---

ex:

++i

We have two types of decrement operators.

1) Post decrement

---

ex:

i--

2) Pre decrement

---

ex:

--i

Post Increment/Decrement

Rule1: First Take
Rule2 : Then Change

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

i++;

System.out.println(i); // 11
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

System.out.println(i++);// 10
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

int j = i++;
```

```
System.out.println(i+" "+j);//11 10
}
}
```

ex:

```
---
class Test
{
public static void main(String[] args)
{
int i = 10;

int j = i-- + i--; // 10 + 9

System.out.println(i+" "+j);//8 19
}
}
```

ex:

```
----
class Test
{
public static void main(String[] args)
{
int i = 10;

int j = i++ + i++;//10 + 11

System.out.println(i+" "+j);//12 21
}
}
```

ex:

```
---
class Test
{
public static void main(String[] args)
```

```
{
int i = 10;

int j = i++ + i++ + i--; //10 + 11 + 12

System.out.println(i+" "+j);//11 33
}
}
```

PRE increment/decrement

Rule1: First Change

Rule2: Then Take

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

++i;

System.out.println(i); // 11
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;

System.out.println(++i); // 11
```

```
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;
int j = ++i;
System.out.println(i+" "+j); // 11 11
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;
int j = ++i + ++i;//11 12
System.out.println(i+" "+j); //12 23
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;
int j = --i + --i + --i;//9 + 8 + 7
System.out.println(i+" "+j); //7 24
}
```

```
}

ex:
---
class Test
{
public static void main(String[] args)
{
int i = 100;
100++;
System.out.println(i); //C.T.E
}
}

ex:
---
class Test
{
public static void main(String[] args)
{
int i = 10;
System.out.println(++(i++)); //C.T.E
}
}

ex:
---
class Test
{
public static void main(String[] args)
{
byte b = 127;

b++;

System.out.println(b); //-128
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 10;
System.out.println(i++ + ++i); // 10 + 12
}
}
```

Control Statements

==========

Control statement enables the programmer to control flow of a program.

Control statement allows us to make decisions, to jump from one section of code to another section and to execute the code repeatedly.

We have three control statements in java.

1) Decision Making Statement

2) Iteration Statement

3) Jump Statement

1) Decision Making Statement

It is used to execute the source code based on the condition.

Decision making statement is possible by using following ways.

i) if stmt

ii) if else stmt

iii) if else if ladder

iv) nested if stmt

v) switch case

i) if stmt

==========

It executes the source code only if our condition is true.

syntax:

---

if(condition)

{

-

- //code to be execute

-

}

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(true)
{
System.out.println("stmt2");
}
System.out.println("stmt3");
}
}
```

o/p:

stmt1

stmt2

stmt3

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(5>20)
{
System.out.println("stmt2");
}
System.out.println("stmt3");
}
}
```

o/p:

stmt1

stmt3

ex:

```
class Test
{
public static void main(String[] args)
{
if(5>20)
System.out.println("stmt1");
System.out.println("stmt2");
System.out.println("stmt3");
}
}
```

o/p:

stmt2

stmt3

Q) Write a java program to find out greatest of two numbers?

```
import java.util.Scanner;
class Test
```

```
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt();
System.out.println("Enter the second number : ");
int b = sc.nextInt();

if(a>b)
System.out.println(a+" is greatest");

if(b>a)
System.out.println(b+" is greatest");
}
}
```

Q) Write a java program to find out greatest of three numbers?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the first number : ");
int a = sc.nextInt();
System.out.println("Enter the second number : ");
int b = sc.nextInt();
System.out.println("Enter the third number : ");
int c = sc.nextInt();

if((a>b) && (a>c))
System.out.println(a+" is greatest");

if((b>a) && (b>c))
System.out.println(b+" is greatest");
```

```
if((c>a) && (c>b))
System.out.println(c+" is greatest");
}
}
```

ii) if else stmt

It is used to execute the source code either our condition is true or false.

syntax:

```
if(condition)
{
- //code to be execute if cond is true
}
else
{
- //code to be execute if cond is false
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(!(5>20))
{
System.out.println("stmt2");
}
else
{
System.out.println("stmt3");
}
System.out.println("stmt4");
}
```

```
}
```
o/p:
stmt1
stmt2
stmt4

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(!(5>2))
{
System.out.println("stmt2");
}
else
{
System.out.println("stmt3");
}
System.out.println("stmt4");
}
}
```
o/p:
stmt1
stmt3
stmt4

Q) Write a java program to find out given age is eligible to vote or not?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
```

```
Scanner sc = new Scanner(System.in);
System.out.println("Enter the age : ");
int age = sc.nextInt();
if(age>=18)
System.out.println("U r eligible to vote");
else
System.out.println("U r not eligible to vote");
}
}
```

Day:26_____ 25-08-26 Tuesday

==========

Q) Write a java program to find out given number is even or odd?

input:
2
output:
It is even number

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();

if (n%2==0)
System.out.println("It is even number");
else
System.out.println("It is odd number");
}
}
```

Q) Write a java program find out given number is positive or negative?

ex:

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();

if(n==0)
{
System.out.println("It is not a positive or negative");
System.exit(0);
}

if(n>0)
System.out.println("It is a positive number");
else
System.out.println("It is a negative number");
}
}
```

Q) John wants to buy new shoes. He visited a big showroom to buy the shoe. But he has a myth that if shoe price divisible by 3 and 5 then only he can buy the shoe. Write a java console to find out John can buy the shoe or not?

Input:

price : 1500

Output:

John can buy the shoe.

```
import java.util.Scanner;
class Test
```

```
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the shoe price : ");
int price = sc.nextInt();

if(price%3==0 && price%5==0)
System.out.println("John can buy the shoe");
else
System.out.println("John can't buy the shoe");
}
}
```

Q) Write a java program to check given alphabet is a vowel or not?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the alphabet : ");
char ch = sc.next().charAt(0);

if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
System.out.println("It is a vowel");
else
System.out.println("It is not a vowel");
}
}
```

Q) Write a java program to find out given year is a leap year or not?

```
import java.util.Scanner;
class Test
{
```

```
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the year : ");
int year = sc.nextInt();

if((year%4==0) && (year%100!=0) || year%400==0)
System.out.println("It is a leap year");
else
System.out.println("It is not a leap year");
}
}
```

iii) if else if ladder

==========

It execute the source code based on multiple conditions.

syntax:

```
if(cond1)
{
-//code to be execute if cond1 is true
}
else if(cond2)
{
-//code to be execute if cond2 is true
}
else if(cond3)
{
-//code to be execute if cond3 is true
}
else
{
-//code to be execute if all conditions are false
}
```

ex:

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the error status code : ");
int status = sc.nextInt();

if(status==200)
System.out.println("OK");
else if(status==404)
System.out.println("Client Side Error");
else if(status==500)
System.out.println("Server Side Error");
else
System.out.println("Unknown Error");
}
}
```

Q) Write a java program to find out given alphabet is a uppercase letter, lowercase letter, digit or special symbol?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the alphabet : ");
char ch = sc.next().charAt(0);

if(Character.isUpperCase(ch))
System.out.println("It is a uppercase letter");
else if(Character.isLowerCase(ch))
System.out.println("It is a lowercase letter");
else if(Character.isDigit(ch))
System.out.println("It is digit");
```

```
else
System.out.println("It is a special symbol");

}
}
```

Q) Write a java program to find out discount based on purchase?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the purchase amount : ");
int amount = sc.nextInt();

double discount=0;

if(amount<=1000)
discount = amount * 0.05;
else if(amount<=2000)
discount = amount * 0.10;
else if(amount<=3000)
discount = amount * 0.20;
else if(amount<=5000)
discount = amount * 0.30;

System.out.println("Amount : "+amount);
System.out.println("Discount : "+discount);
System.out.println("Actual Amount : "+(amount-discount));
}
}
```

Assignment

==========

Q) Write a java program to find out given alphabet is a vowel or not using if else if ladder?

Q) Write a java program to accept six marks of a student then find out total,average and grade? if average is greater then equals to 70 then A grade, If average is greater then equals to 50 B grade, If average is greater then equals to 35 C grade. if average is less then 35 then failed?

iv) nested if stmt

==========

If stmt contains another if stmt is called nested if stmt.

syntax:

```
if(condition)
{
if(condition)
{
- //code to be execute
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(5>2)
{
System.out.println("stmt2");
if(true)
{
System.out.println("stmt3");
}
System.out.println("stmt4");
}
System.out.println("stmt5");
}
}
```

o/p:

stmt1
stmt2
stmt3
stmt4
stmt5

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(5>20)
{
System.out.println("stmt2");
if(true)
{
System.out.println("stmt3");
}
System.out.println("stmt4");
}
System.out.println("stmt5");
}
}
```
o/p:
stmt1
stmt5

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(5>2)
{
```

```
System.out.println("stmt2");
if(false)
{
System.out.println("stmt3");
}
System.out.println("stmt4");
}
System.out.println("stmt5");
}
}
```

o/p:

stmt1

stmt2

stmt4

stmt5

Q) Write a java program to find out given number is positive or negative?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();

if(n!=0)
{
if(n>0)
{
System.out.println("It is a positive number");
System.exit(0);
}
System.out.println("It is a negative number");
}
}
}
```

==========26th august holiday==========

Day:27_____ 27-08-26 Thursday

==========

Q) Write a java program to find out the cost based on user inputs. A program must prompt users to enter total weight of items (in kilograms) and shipping destination (domestic or international). For domestic orders, a program must charge Rs.500 upto 5kg and Rs.100 per additional kg.For international orders a program must charge Rs.1000 upto 5kg and Rs.200 per additional kg, and Rs.500 surcharge if weight exceeded to 10 kg.Find out or calculated shipping cost.

Input:

Enter total weight of items : 11

Enter shipping destination : domestic

Output:

Shipping Cost : 1100

Input:

Enter total weight of items : 11

Enter shipping destination : iternational

Output:

Shipping Cost : 2700

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the total weight of items : ");
int weight = sc.nextInt(); // 11
System.out.println("Enter the shipping destination : ");
String destination = sc.next().toLowerCase(); // domestic
```

```
int cost = 0;
if(destination.equals("domestic"))
{
if(weight<=5)
cost = 500;
else
cost = 500 + (weight-5)*100;
}
else if(destination.equals("international"))
{
if(weight<=5)
cost = 1000;
else if(weight>5 && weight<=10)
cost = 1000 + (weight-5)*200;
else
cost = 1000 + (weight-5)*200 + 500;
}
else
{
System.out.println("Invalid destination");
return;
}

System.out.println("SHipping Cost : "+cost);
}
}
```

Switch case

==========

It is used to execute the source code based on multiple conditions.

It is similar to if else if ladder.

To read options from enduser we need to use switch case.

syntax:

```
switch(condition/expression)
{
case value1:
//code to be execute
break;
case value2:
//code to be execute
break;
-
-
default:
//code to be execute if all cases are false
}
```

Q) Write a java program to accept one option and display it details?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the option : ");
int option = sc.nextInt();

switch(option)
{
case 100: System.out.println("It is a police number");
break;
case 103: System.out.println("It is a enquiry number");
break;
case 108: System.out.println("It is a emergency number");
break;
default: System.out.println("Invalid option");
}
}
}
```

Declaration of break statement is optional. If we define break statement then from where our condition is satisfied from there all cases will be executed that state is called fall through state of switch case.

ex:

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the option : ");
int option = sc.nextInt();

switch(option)
{
case 100: System.out.println("It is a police number");
//break;
case 103: System.out.println("It is a enquiry number");
//break;
case 108: System.out.println("It is a emergency number");
//break;
default: System.out.println("Invalid option");
}
}
}
```

The allowed datatype for switch case are byte,short,char,int and string.

Q)Write a java program to find out given alphabet is a vowel or consonent?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the alphabet :");
```

```
char ch = sc.next().charAt(0);

//convert character to lowercase
ch = Character.toLowerCase(ch);

switch(ch)
{
case 'a','e','i','o','u' -> System.out.println("It is a vowel");
default -> System.out.println("It is a consonent");
}
}
}
```

yield
=======
A yield keyword introduced in Java 14.

It is used to return a value from case block within a switch expression.

ex:
--

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the day of a week :");
int day = sc.nextInt();

String result = switch(day)
{
case 1,2,3,4,5 -> { yield "Weakday"; }
case 6,7 -> { yield "Weekend"; }
default -> { yield "Invalid Day"; }
};
System.out.println(result);
```

}
}

2) Iteration Statements
==========
Iteration statement is used to execute the code repeatedly.

Iteration statement is possible using loops.

We have four types of loops in java.

i) do while loop

ii) while loop

iii) for loop

iv) for each loop

i) do while loop

It is used to execute the source code how long our condition is true.

syntax:
---
do
{
-
- //code to be execute
-
}while(condition);

ex:
---
class Test
{
public static void main(String[] args)

```
{
int i = 1;
do
{
System.out.print(i+" ");//infinite 1
}
while (i<=10);
}
}
```

In do while loop, our code will execute atleast for one time either our condition is true or false.

ex:

```
class Test
{
public static void main(String[] args)
{
int i = 11;
do
{
System.out.print(i+" "); // 11
}
while (i<=10);
}
}
```

Q) Write a java program to display 10 natural numbers?

```
class Test
{
public static void main(String[] args)
{
int i = 1;
do
{
```

```
System.out.print(i+" ");//1 2 3 4 5 6 7 8 9 10
i++;
}
while (i<=10);
}
}
```

Q) Write a java program to find out sum of 10 natural numbers?

```
class Test
{
public static void main(String[] args)
{
int i = 1, sum = 0;
do
{
sum = sum + i;
i++;
}
while (i<=10);
System.out.println(sum);
}
}
```

Q) Write a java program to find out factorial of a given number?

input:
5

output:
120 (5*4*3*2*1)

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
```

```
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//5

int i = n, fact = 1;
do
{
fact = fact * i;
i--;
}
while (i>=1);

System.out.println(fact);
}
}
```

Q) Write a java program to find out multiplication table of a given number?

input:

5

output:

5 * 1 = 5

-

-

5 * 10 = 50

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//5

int i = 1;
do
```

```
{
System.out.println(n+" * "+i+" = "+n*i);
i++;
}
while (i<=10);
}
}
```

Day:29_____ 28-08-26 Friday

==========

ii) while loop

==========

It is used to execute the source code how long our condition is true.

syntax:

```
while(condition)
{
-
- //code to be execute
-
}
```

ex:

--

```
class Test
{
public static void main(String[] args)
{
int i=1;
while(i<=10)
{
System.out.print(i+" ");//infinite 1
}
}
```

```
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int i=11;
while(i<=10)
{
System.out.print(i+" ");// no output
}
}
}
```

Q) Write a java program to display 10 natural numbers using while loop?

```
class Test
{
public static void main(String[] args)
{
int i=1;
while(i<=10)
{
System.out.print(i+" "); // 1 2 3 4 5 6 7 8 9 10
i++;
}
}
}
```

Q) Write a java program to perform sum of 10 natural numbers?

```
class Test
{
public static void main(String[] args)
{
```

```
int i=1,sum=0;
while(i<=10)
{
sum = sum + i;
i++;
}
System.out.println(sum);
}
}
```

Q) Write a java program to find out factorial of a given number?

Input:
5

Output:
120 (5*4*3*2*1)

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//5

int i=n, fact=1;
while(i>=1)
{
fact = fact * i;
i--;
}
System.out.println(fact);
}
}
```

Q) Write a java program to find out sum of digits of a given number?

input:
123
output:
6

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//123

int rem,sum=0;
while(n>0)
{
rem = n%10;
sum = sum + rem;
n = n/10;
}
System.out.println(sum);
}
}
```

Q) Write a java program to find out given number is armstrong or not?

input:
153

output:

It is a armstrong number
1*1*1+5*5*5+3*3*3 -> 1+125+27 -> 153

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//123
int temp = n;
int rem,sum=0;
while(n>0)
{
rem = n%10;
sum = sum + rem*rem*rem;
n = n/10;
}
if(temp==sum)
System.out.println("It is a armstrong number");
else
System.out.println("It is not a armstrong number");
}
}
```

Q) Write a java program to display reverse of a given number?

input:
123
output:
321

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
```

```
System.out.println("Enter the number : ");
int n = sc.nextInt();//123

int rem,rev=0;
while(n>0)
{
rem = n%10;
rev = rev*10+rem;
n = n/10;
}
System.out.println(rev);
}
}
```

Q) Write a java program to find out given number is palindrome or not?

input:
121
output:
It is a palindrome number

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//123
int temp = n;
int rem,rev=0;
while(n>0)
{
rem = n%10;
rev = rev*10+rem;
n = n/10;
}
```

```
if(temp==rev)
System.out.println("It is a palindrome number");
else
System.out.println("It is not a palindrome number");
}
}
```

Q) A Frog felldown in a well of 10 meters. Each day frog tries to come out from the well. Frog jumps two meters and rollback to one meters. Write a java program in how many days a frog can come out from the well.

```
class Test
{
public static void main(String[] args)
{
int wellLength = 10;
int jumps = 2;
int rollback = 1;

int days=0;
int currentPosition = 0;
while(currentPosition<wellLength)
{
currentPosition+=jumps;
if(currentPosition>wellLength)
{
break;
}
currentPosition-=rollback;
days++;
}
System.out.println("Frog will come out from well in "+days+" days");
}
}
```

iii) for loop

==========

It is used to execute the source code how long our condition is true.

syntax:

---

for(initialization;condition;increment/decrement)

{

-

- //code to be execute

-

}

If number of iterations are known by the user then we need to use for loop.

If number of iterations are not known by the user then we need to use while loop.

If number of iterations are not known by the user but code must execute atleast for one time then we need to use do while loop.

ex:

--

```
class Test
{
public static void main(String[] args)
{
for(int i=1;i<=10;i++)
{
System.out.print(i+" "); //1 2 3 4 5 6 7 8 9 10
}
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
for(int i=1;i<=10;i++)
```

```
{
if(i%2==0)
System.out.print(i+" "); //2 4 6 8 10
}
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
for(int i=1;i<=10;i++)
{
if(i%2==0)
{
System.out.print(i+" "); // 2 6 10
i+=2;
}
}
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int cnt = 0;
for(int i=1;i<=10;i++)
{
if(i%2==0)
{
cnt++;
}
}
```

```
System.out.println(cnt);//5
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
for(;;)
{
System.out.print("Hello ");
}
}
}
```

o/p:

infinite Hello

Q) Write a java program to find out given number prime or not?

Input:

5

Output:

It is a prime number

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number :");
int n = sc.nextInt();//5

boolean flag = true;
```

```
for(int i=2;i<=n/2;i++)
{
if(n%i==0)
{
flag=false;
break;
}
}
if(flag==true)
System.out.println("It is a prime number");
else
System.out.println("It is not a prime number");
}
}
```

Day:30_____ 29-08-26 Saturday

==========

Q) Write a java program to display prime numbers from 1 to 100?

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, and 97.

ex:

```
class Test
{
public static void main(String[] args)
{
for(int n=2;n<=100;n++)
{
boolean flag=true;
for(int i=2;i<=n/2;i++)
{
if(n%i==0)
{
flag = false;
```

```
break;
}
}
if (flag==true)
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to find out fibonacci series of a given number?

Input:
5

output:
0 1 1 2 3

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//5

int a=0,b=1,c;
for(int i=1;i<=n;i++)
{
System.out.print(a+" ");
c = a + b;
a = b;
b = c;
}
}
}
```

Q) Write a java program to find out given number is perfect or not?

input:
6

output:
It is a perfect number

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//6

int sum=0;
for(int i=1;i<n;i++)
{
if(n%i==0)
{
sum+=i;
}
}
if(n==sum)
System.out.println("It is a perfect number");
else
System.out.println("It is not a perfect number");

}
}
```

Q) Write a java program to find out GCD(Greatest Common Divisor) of two numbers?

Input:

12 18

Output:
6

```
class Test
{
public static void main(String[] args)
{
int a=12,b=18,gcd=0;

for(int i=1;i<=a && i<=b;i++)
{
if((a%i==0) && (b%i==0))
{
gcd = i;
}
}
System.out.println("GCD of two numbers is = "+gcd);
}
}
```

iv) for each loop
==========
It is introduced in Java 1.5.

for each loop is also known as enhanced for loop.

It is used to iterate the elements/objects from arrays/collections.

syntax:

```
for (dataType variableName : arrayOrCollection)
{
// Code block to be executed
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
int[] arr = {10,20,30,40};
for(int i : arr)
{
System.out.print(i+" ");
}
}
}
```

ex:

```
import java.util.*;
class Test
{
public static void main(String[] args)
{
List<Integer> list = List.of(1,7,2,9,5);
for (int i : list)
{
System.out.print(i+" ");
}
}
}
```

3) Jump Statement

==========

Jump statement is used to jump from one section of code to another section.

It transfer total control to the sub part of a program.

We have three jump statements in java.

i) break stmt

ii) continue stmt

iii) return stmt

i) break stmt

==========

It is used to break the execution of loops and switch case.

For conditional statements we can use if condition.

syntax:

---

break;

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
break;
System.out.println("stmt2");
}
}
```

o/p:

C.T.E : break outside switch or loop

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(false)
```

```
{
break;
}
System.out.println("stmt2");
}
}
```

o/p:

C.T.E : break outside switch or loop

ex:

```
class Test
{
public static void main(String[] args)
{
for(int i=1;i<=10;i++)
{
if(i==5)
{
break;
}
System.out.print(i+" ");//1 2 3 4
}
}
}
```

ii) continue stmt

It is used to continue the execution of loops.

For conditional statements we can use if condition.

It is used to skip the current iteration and moves to next iteration.

syntax:

```
continue;
```

ex:

```
class Test
```

```
{
public static void main(String[] args)
{
System.out.println("stmt1");
continue;
System.out.println("stmt2");
}
}
```

o/p:

C.T.E : continue outside of loop

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println("stmt1");
if(false)
{
continue;
}
System.out.println("stmt2");
}
}
```

o/p:

C.T.E : continue outside of loop

ex:

```
class Test
{
public static void main(String[] args)
{
for(int i=1;i<=10;i++)
{
if(i==5)
{
```

```
continue;
}
System.out.print(i+" ");//1 2 3 4 6 7 8 9 10
}
}
}
```

iii) return stmt

It is used to exit from methods.

It contains optional value.

syntax:

optional

|

return <value>;

iii) return stmt

==========

It is used to exit from methods.

It contains optional value.

syntax:

---

optional

|

return <value>;

ex:

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter error status code : ");
int code = sc.nextInt();

if(code==200)
System.out.println("OK");
else if(code==404)
System.out.println("Client Side Error");
else if(code==500)
System.out.println("Server Side Error");
else
return;
}
}
```

Loop Patterns

==========

1)

```
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=4;j++)
```

```
{
System.out.print(i+" ");
}
//new line
System.out.println();
}
}
}
```

2)

```
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=4;j++)
{
System.out.print(j+" ");
}
//new line
System.out.println();
}
}
}
```

3)

```
* * * *
* * * *
* * * *
```

* * * *

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=4;j++)
{
System.out.print("* ");
}
//new line
System.out.println();
}
}
}
```

4)
4 4 4 4
3 3 3 3
2 2 2 2
1 1 1 1

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=4;i>=1;i--)
{
//cols
for(int j=1;j<=4;j++)
{
System.out.print(i+" ");
```

```
}
//new line
System.out.println();
}
}
}
```

5)

```
* * * *
* *
* *
* * * *
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=4;j++)
{
if(i==1 || i==4 || j==1 || j==4)
System.out.print("* ");
else
System.out.print(" ");
}
//new line
System.out.println();
}
}
}
```

6)

```
* - - -
- * - -
- - * -
- - - *
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=4;j++)
{
if(i==j)
System.out.print("* ");
else
System.out.print("- ");
}
//new line
System.out.println();
}
}
}
```

7)

```
* - - - *
- * - * -
- - * - -
- * - * -
* - - - *
```

```
class Test
{
public static void main(String[] args)
{
```

```
//rows
for(int i=1;i<=5;i++)
{
//cols
for(int j=1;j<=5;j++)
{
if(i==j || i+j==6)
System.out.print("* ");
else
System.out.print("- ");
}
//new line
System.out.println();
}
}
}
```

8)

```
*
*
* * * * *
*
*
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=5;i++)
{
//cols
for(int j=1;j<=5;j++)
{
if(i==3 || j==3)
System.out.print("* ");
else
System.out.print(" ");
```

```
}
//new line
System.out.println();
}
}
}
```

9)

```
4 4 4 4 4 4 4
4 3 3 3 3 3 4
4 3 2 2 2 3 4
4 3 2 1 2 3 4
4 3 2 2 2 3 4
4 3 3 3 3 3 4
4 4 4 4 4 4 4
```

```
class Test
{
public static void main(String[] args)
{
int size = 7;
int n = 4;
for(int i=0;i<size;i++)
{
for(int j=0;j<size;j++)
{
int result = n - Math.min(Math.min(i,j), Math.min(size-i-1,size-j-1));
System.out.print(result+" ");
}
//new line
System.out.println();
}
}
}
```

Left side pattern

==========

1)

1
2 2
3 3 3
4 4 4 4

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=i;j++)
{
System.out.print(i+" ");
}
//new line
System.out.println();
}
}
}
```

2)

4 4 4 4
3 3 3
2 2
1

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=4;i>=1;i--)
```

```
{
//cols
for(int j=1;j<=i;j++)
{
System.out.print(i+" ");
}
//new line
System.out.println();
}
}
}
```

3)

```
*
* *
* * *
* * * *

class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=i;j++)
{
System.out.print("* ");
}
//new line
System.out.println();
}
}
}
```

4)

1
1 # 2
1 # 2 # 3
1 # 2 # 3 # 4

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=i;j++)
{
if(j==1)
System.out.print(j);
else
System.out.print(" # "+j);
}
//new line
System.out.println();
}
}
}
```

5)
2
4 6
8 10 12
14 16 18 20

```
class Test
{
public static void main(String[] args)
```

```
{
int n = 2;
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=i;j++)
{
System.out.print(n+" ");
n+=2;
}
//new line
System.out.println();
}
}
}
```

6)

```
1
2 3
4 5 6
7 8 9 0
```

```
class Test
{
public static void main(String[] args)
{
int n = 1;
//rows
for(int i=1;i<=4;i++)
{
//cols
for(int j=1;j<=i;j++)
{
if(n<=9)
System.out.print(n++ +" ");
else
System.out.print("0 ");
```

```
}
//new line
System.out.println();
}
}
}
```

Assignment

---

1)

```
*
* *
* * *
* * * *
* * *
* *
*
```

Day:32_____ 01-09-26 Tuesday

==========

7)

```
2
3 5
7 11 13
17 19 23 29
```

```
class Test
{
public static void main(String[] args)
{
int n = 2;

for(int i=1;i<=4;i++)
{
for(int j=1;j<=i;j++)
{
```

```
while(true)
{
boolean flag = true;
for(int k=2;k<=n/2;k++)
{
if(n%k==0)
{
flag = false;
break;
}
}
if(flag==true)
{
System.out.print(n+" ");
n++;
break;
}
else
{
n++;
}
}
}
//new line
System.out.println();
}
}
}
```

Right Side Patterns

==========

1)

```
1
2 2
3 3 3
4 4 4 4
```

```
class Test
```

```
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//right side
for(int j=1;j<=i;j++)
{
System.out.print(i+" ");
}
//new line
System.out.println();
}
}
}
```

2)

```
4 4 4 4
3 3 3
2 2
1
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=4;i>=1;i--)
{
//space
```

```
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//right side
for(int j=1;j<=i;j++)
{
System.out.print(i+" ");
}
//new line
System.out.println();
}
}
}
```

3)

```
*
* *
* * *
* * * *
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//elements
for(int j=1;j<=i;j++)
{
System.out.print("* ");
```

```
}
//new line
System.out.println();
}
}
}
```

Pyramid patterns

==========

1)

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
System.out.print(j+" ");
}
//right side
for(int j=i-1;j>=1;j--)
{
System.out.print(j+" ");
}
//new line
```

```
System.out.println();
}
}
}
```

2)

```
1 2 3 4 3 2 1
1 2 3 2 1
1 2 1
1
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=4;i>=1;i--)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
System.out.print(j+" ");
}
//right side
for(int j=i-1;j>=1;j--)
{
System.out.print(j+" ");
}
//new line
```

```
System.out.println();
}
}
}
```

3)

```
*
* * *
* * * * *
* * * * * * *
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
System.out.print("* ");
}
//right side
for(int j=i-1;j>=1;j--)
{
System.out.print("* ");
}
//new line
System.out.println();
}
}
```

}

Q) Write a java program to display below loop pattern?

1 1
1 2 2 1
1 2 3 3 2 1
1 2 3 4 4 3 2 1

```
class Test
{
public static void main(String[] args)
{
int rows = 4;
for(int i=1;i<=rows;i++)
{
//left side
for(int j=1;j<=i;j++)
{
System.out.print(j+" ");
}
//space
for(int j=1;j<=(rows-i)*2;j++)
{
System.out.print(" ");
}

//right side
for(int j=i;j>=1;j--)
{
System.out.print(j+" ");
}
//new line
System.out.println();
}
}
}
```

Q) Write a java program to display below loop pattern?

6 5 4 3 2 1
6 5 4 3 2
6 5 4 3
6 5 4
6 5
6

```
class Test
{
public static void main(String[] args)
{
for(int i=1;i<=6;i++)
{
//space
for(int j=1;j<i;j++)
{
System.out.print(" ");
}
//elements
for(int j=6;j>=i;j--)
{
System.out.print(j+" ");
}
//new line
System.out.println();
}
}
}
```

Q) Write a java program to display pascal triangle?

1
1 1

1 2 1
1 3 3 1
1 4 6 4 1

```
class Test
{
public static void main(String[] args)
{
for(int i=0;i<5;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//elements
int result = 1;
for(int k=0;k<=i;k++)
{
System.out.print(result+" ");
result = result * (i-k)/(k+1);
}
//new line
System.out.println();
}
}
}
```

Q) Write a java program to display below loop pattern?

*
* *
* * *
* * * *
* * * * *

```
class Test
{
public static void main(String[] args)
{
for(int i=0;i<5;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//elements
for(int k=0;k<=i;k++)
{
System.out.print("* ");
}
//new line
System.out.println();
}
}
}
```

Assignment

==========

Q) Write a java program to display diamond loop pattern?

```
*
* * *
* * * * *
* * * * * * *
* * * * *
* * *
*
```

```
class Test
{
public static void main(String[] args)
{
```

```
//rows
for(int i=1;i<=4;i++)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
System.out.print("* ");
}
//right side
for(int j=i-1;j>=1;j--)
{
System.out.print("* ");
}
//new line
System.out.println();
}

//rows
for(int i=3;i>=1;i--)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
System.out.print("* ");
}
//right side
for(int j=i-1;j>=1;j--)
{
```

```
System.out.print("* ");
}
//new line
System.out.println();
}
}
}
```

2)

```
*
* *
* *
* *
* *
* *
*
```

```
class Test
{
public static void main(String[] args)
{
//rows
for(int i=1;i<=4;i++)
{
//space
for(int j=4;j>i;j--)
{

System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
if(j==1)
System.out.print("* ");
```

```
else
System.out.print(" ");
}
//right side
for(int j=i-1;j>=1;j--)
{
if(j==1)
System.out.print("* ");
else
System.out.print(" ");
}
//new line
System.out.println();
}

//rows
for(int i=3;i>=1;i--)
{
//space
for(int j=4;j>i;j--)
{
System.out.print(" ");
}
//left side
for(int j=1;j<=i;j++)
{
if(j==1)
System.out.print("* ");
else
System.out.print(" ");
}
//right side
for(int j=i-1;j>=1;j--)
{
if(j==1)
System.out.print("* ");
else
System.out.print(" ");
```

```
}
//new line
System.out.println();
}
}
}
```

Day:33_____ 02-09-26 Wednesday

==========

Various ways to declare methods in java

==========

There are four ways to declare methods in java.

1) No returntype with No argument method

2) No returntype with argument method

3) With returntype with No argument method

4) With returntype with argument method

1) No returntype with No argument method

If we don't have arguments then we need to read input values inside callie method.

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
//caller method
sum();
}
//callie method
public static void sum()
```

```
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the two numbers :");
int a = sc.nextInt();
int b = sc.nextInt();
int c = a + b;
System.out.println("sum of two numbers is = "+c);
}
}
```

2) No returntype with argument method

If we have arguments then we need to read input values inside main method.
Number of inputs must be equals to number of arguments.

ex:

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the two numbers :");
int a = sc.nextInt();//20
int b = sc.nextInt();//30

//caller method
sum(a,b);
}
//callie method
public static void sum(int a,int b)
{
int c = a + b;
System.out.println("sum of two numbers is ="+c);
}
}
```

3) With returntype with No argument method

At a time we can return only one value.

A return type is completely depend upon output datatype.

ex:

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
//caller method
int k = sum();
System.out.println("sum of two numbers is = "+k);
}
//callie method
public static int sum()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the two numbers :");
int a = sc.nextInt();//20
int b = sc.nextInt();//30
int c = a + b;
return c;
}
}
```

4) With returntype with argument method

We can declare multiple return statements but we can return only one value at a time.

If we have single output then only we need to use return type.

ex:

```
import java.util.Scanner;
```

```
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the two numbers :");
int a = sc.nextInt();//20
int b = sc.nextInt();//30
//caller method
System.out.println("sum of two numbers is = "+sum(a,b));
}
//callie method
public static int sum(int a,int b)
{
int c = a + b;
return c;
}
}
```

Assignment

==========

Q) Write a java program to check given number is even or odd using four ways?

Recursion

==========

It is a programming technique which breaks down complex problem into sub-problems.

A method which called itself for many number of times to solve a problem is called recursion.

Recursion is similar to loopings. Whenever we use recursion we should not use loops.

Q) Write a java program to display 10 naturals without using loops?

```
class Test
```

```
{
public static void main(String[] args)
{
//caller method
display(1);
}
//callie method
public static void display(int i)
{
if(i<=10)
{
System.out.print(i+" ");//1 2 3 4 5 6 7 8 9 10

display(i+1);
}
}
}
```

Q) Write a java program to perform sum of two numbers without using arithmetic operator?

```
class Test
{
public static void main(String[] args)
{
//caller method
System.out.println(sum(5,10));
}
//callie method
public static int sum(int a,int b)
{
if(a==0)
return b;

return sum(--a,++b);
}
}
```

Q) Write a java program to find out factorial of a given number using recursion?

input:
5

output:
120

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//5

//caller methohd
System.out.println(factorial(n));
}
//callie method
public static int factorial(int n)
{
if(n<0)
return -1;
if(n==0)
return 1;
return n*factorial(n-1);
}
}
```

Q) Write a java program to find out N-th element of fibonacci series?

fibonacci series : 0 1 1 2 3 5 .....

input:
4

output:
2

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number : ");
int n = sc.nextInt();//4

//caller methohd
System.out.println(fib(n));
}
//callie method
public static int fib(int n)
{
if(n==0 || n==1)
return 0;
if(n==2)
return 1;

return fib(n-1)+fib(n-2);
}
}
```

Day:34_____ 03-09-26 Thursday
==========

Arrays
=======
A normal variable can hold one value at a time.

To store multiple values in a single variable then we need to use arrays.

Arrays is a collection of homogeneous data elements.

The main advantages of arrays are

1) We can represent multiple elements using single variable name.
ex:
int[] arr={10,20,30};

2) Performance point of view arrays are recommended to use.

The main disadvantages of arrays are

1) It is fixed in size. Once if we create an array there is no chance of increasing or decreasing the size of an array.

2) To use array concept in advanced we should know what is the size of an array which is always not possible.

In java, arrays are classified into three types.

1) Single Dimensional Array

2) Double Dimensional Array

3) Multi-Dimensional Array

Array Declaration

At the time of array declaration we should not specify array size.

Arrays
|---|---|
Single Dimensional Array Double Dimensional Array Multi-Dimensional Array

> int[] arr; int[][] arr; int[][][] arr;
> int []arr; int [][]arr; int [][][]arr;
> int arr[]; int arr[][]; int arr[][][];
int[] []arr; int[][] []arr;

int[] arr[]; int[][] arr[];
int []arr[]; int[] [][]arr;
int[] arr[][];
int[] []arr[];
int [][]arr[];
int []arr[][];

Array Creation

In java, every array consider as an object. Hence we will use new operator/key to create an array.
ex:
int[] arr = new int[3];

Rules to constructor an array

Rule1:

At the time of array creation compulsory we need to specify array size.
ex:
int[] arr = new int[3];
int[] arr = new int[]; // C.T.E array dimension missing

Rule2:

It is legal to have an array size with zero.
ex:
int[] arr = new int[0];
System.out.println(arr.length);
Rule3:

We can't take negative number as an array size otherwise we will get
runtime exception called NegativeArraySizeException.
ex:
int[] arr = new int[-3];

Rule4:

The allowed datatype for an array size is byte,short,int and char.
If we take other datatypes then we will get compile time error.
ex:
byte b = 10;
int[] arr = new int[b];

ex:
int[] arr = new int['a'];

ex:
int[] arr = new int[10.5d]; // C.T.E

Rule5:

The maximum length we can take for an array size is maximum length of integer.
ex:
int[] arr = new int[2147483647];

Array initialization

Whenever we create an array , every array element is initialized with default values.

If we are not happy with default values then we can change with customized values.

Diagram: class26.1

Array Declaration, Creation and Initialization using single line
---

int[] arr;
arr = new int[3];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30; ==> int[] arr = {10,20,30};
==> String[] sarr = {"Hi","Hello","Bye"};
==> char[] carr = {'a','b','c'};

Q) What is the difference between length and length() ?

length

It is a final variable which is applicable only for arrays.
It returns size of an array.
ex:
int[] arr = new int[3];
System.out.println(arr.length); // 3

length()

It is a predefined method which is applicable only for String objects.
It returns number of characters present in a string.
ex:
String str = "raisetech";
System.out.println(str.length());

Single Dimensional Array Programs
==========

Q) Write a java program to accept array elements and display them?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the array size : ");
int size = sc.nextInt(); // 4

int[] arr = new int[size];
```

```
//insert elements
for(int i=0;i<arr.length;i++)
{
System.out.println("Enter the array element : ");
arr[i] = sc.nextInt();
}

//print elements
for(int i=0;i<arr.length;i++)
{
System.out.print(arr[i]+" ");
}
}
}
```

Q) Write a java program to display array elements?

input:
7 2 9 1 5 4

```
class Test
{
public static void main(String[] args)
{
int[] arr = {7,2,9,1,5,4};

for(int i=0;i<arr.length;i++)
{
System.out.print(arr[i]+" ");
}

//new line
System.out.println();

for(int n: arr)
{
```

```
System.out.print(n+" ");
}

}
}
```

Q) Write a java program to display the array elements in reverse order?

input:
7 2 9 1 5 4
output:
4 5 1 9 2 7

```
class Test
{
public static void main(String[] args)
{
int[] arr = {7,2,9,1,5,4};

for(int i=arr.length-1;i>=0;i--)
{
System.out.print(arr[i]+" ");
}
}
}
```

Q) Write a java program to copy the array elements?

input:
7 2 9 1 5 4

ex:

```
class Test
{
public static void main(String[] args)
```

```
{
int[] arr = {7,2,9,1,5,4};

int[] newArr = new int[arr.length];

newArr = arr;

for(int n : newArr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to perform sum of array elements?

input:
7 2 9 1 5 4

output:
28

```
class Test
{
public static void main(String[] args)
{
int[] arr = {7,2,9,1,5,4};
int sum = 0;
for(int n : arr)
{
sum += n;
}
System.out.println(sum);
}
}
```

Q) Write a java program to display even elements from given array ?

input:
7 2 9 6 1 5 4
output:
2 6 4

```
class Test
{
public static void main(String[] args)
{
int[] arr = {7,2,9,6,1,5,4};

for(int n : arr)
{
if(n%2==0)
{
System.out.print(n+" ");
}
}
}
}
```

Q) Write a java program to display prime elements from given array?

input:
8 2 9 7 5 3 4

output:
2 7 5 3

```
class Test
{
public static void main(String[] args)
{
int[] arr = {8,2,9,7,5,3,4};
```

```
for(int n : arr)
{
boolean flag = true;
for(int i=2;i<=n/2;i++)
{
if(n%i==0)
{
flag = false;
break;
}
}
if(flag == true)
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display array elements in sorting order?

input:
8 2 9 7 5 3 4

output:
2 3 4 5 7 8 9

```
import java.util.Arrays;
class Test
{
public static void main(String[] args)
{
int[] arr = {8,2,9,7,5,3,4};

Arrays.sort(arr);

for(int n : arr)
{
System.out.print(n+" ");
```

```
}
}
}
```

Q) Write a java program to display array elements in sorting order without using sort() method?

input:
8 2 9 7 5 3 4

output:
2 3 4 5 7 8 9

```
class Test
{
public static void main(String[] args)
{
int[] arr = {8,2,9,7,5,3,4};

for(int i=0;i<arr.length-1;i++)
{
if(arr[i] > arr[i+1])
{
int temp = arr[i];
arr[i] = arr[i+1];
arr[i+1] = temp;

i=-1;
}
}

//pring elements
for(int n : arr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display array elements in descending order without using sort() method?

input:

8 2 9 7 5 3 4

output:

9 8 7 5 4 3 2

```
class Test
{
public static void main(String[] args)
{
int[] arr = {8,2,9,7,5,3,4};

for(int i=0;i<arr.length-1;i++)
{
if(arr[i] < arr[i+1])
{
int temp = arr[i];
arr[i] = arr[i+1];
arr[i+1] = temp;

i=-1;
}
}

//pring elements
for(int n : arr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display highest element from given array without using sort() method?

input:
8 2 9 7 5 3 4

output:
9

```
class Test
{
public static void main(String[] args)
{
int[] arr = {8,2,9,7,5,3,4};

int big = arr[0];

for(int n : arr)
{
if(n>big)
{
big = n;
}
}
System.out.println(big);
}
}
```

Q) Write a java program to display least element from given array without using sort() method?

input:
8 2 9 7 5 3 4

output:
2

```
class Test
{
public static void main(String[] args)
{
int[] arr = {8,2,9,7,5,3,4};

int small = arr[0];

for(int n : arr)
{
if(n<small)
{
small = n;
}
}
System.out.println(small);
}
}
```

Day:35_____ 04-09-26 Friday

==========

Q) Write a java program to reverse the array without using another array?

Input:
4 9 1 3 6 2

Output:
2 6 3 1 9 4

```
class Test
{
public static void main(String[] args)
{
int[] arr = {4,9,1,3,6,2};
```

```
int left = 0;
int right = arr.length-1;

for(int i=0;i<arr.length;i++)
{
if(left<right)
{
int temp = arr[left];
arr[left] = arr[right];
arr[right] = temp;

left++;
right--;
}
}
for(int n : arr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to merge two arrays and display them in sorting order?

input:
5 1 3 2 4
9 6 8 7 10
output:
1 2 3 4 5 6 7 8 9 10

```
import java.util.Arrays;
class Test
{
public static void main(String[] args)
{
int[] arr1 = {5,1,3,2,4};
```

```
int[] arr2 = {9,6,8,7,10};

int size1 = arr1.length; //5
int size2 = arr2.length; //5

arr1 = Arrays.copyOf(arr1,size1+size2);

int index=0;
for(int i=size1;i<arr1.length;i++)
{
arr1[i] = arr2[index++];
}

Arrays.sort(arr1);

for(int n : arr1)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display three highest elements from given array?

input:
6 2 7 5 8 1 3 9

output:
9 8 7

```
class Test
{
public static void main(String[] args)
{
int[] arr = {6,2,7,5,8,1,3,9};
```

```
int firstElement = Integer.MIN_VALUE;
int secondElement = Integer.MIN_VALUE;
int thirdElement = Integer.MIN_VALUE;

for(int n : arr)
{
if(n>firstElement)
{
thirdElement = secondElement;
secondElement = firstElement;
firstElement = n;
}
else if(n>secondElement)
{
thirdElement = secondElement;
secondElement = n;
}
else if(n>thirdElement)
{
thirdElement = n;
}
}
System.out.println(firstElement+" "+secondElement+" "+thirdElement);
}
}
```

Q) Write a java program to seggregate array elements without using sort() method?

input:
1 0 0 1 1 0 1 0 0 1

output:
0 0 0 0 0 1 1 1 1 1

```
class Test
{
```

```
public static void main(String[] args)
{
int[] arr = {1,0,0,1,1,0,1,0,0,1};

int index = 0;
for(int i=0;i<arr.length;i++)
{
if(arr[i]==0)
{
int temp = arr[i];
arr[i] = arr[index];
arr[index] = temp;
index++;
}
}

for(int n : arr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display unique elements from given array?

input:
4 8 2 9 1 8 7 7 6 2

output:
4 9 1 6

```
class Test
{
public static void main(String[] args)
{
int[] arr = {4,8,2,9,1,8,7,7,6,2};
```

```
for(int i=0;i<arr.length;i++)
{
int cnt = 0;
for(int j=0;j<arr.length;j++)
{
if(arr[i] == arr[j])
{
cnt++;
}
}
if(cnt==1)
System.out.print(arr[i]+" ");
}
}
}
```

Q) Write a java program to display duplicate elements from given array?

input:
4 8 2 9 1 8 7 7 6 2

output:
2 8 7

```
class Test
{
public static void main(String[] args)
{
int[] arr = {4,8,2,9,1,8,7,7,6,2,8};

for(int i=0;i<arr.length;i++)
{
int cnt = 0;
for(int j=i+1;j<arr.length;j++)
{
if(arr[i] == arr[j])
```

```
{
cnt++;
}
}
if(cnt==1)
System.out.print(arr[i]+" ");
}
}
}
```

Q) Write a java program to display most repeating element from given array?

input:
2 7 1 2 5 5 2 9 3 2 8

output:
2 repeating for 4 times

```
class Test
{
public static void main(String[] args)
{
int[] arr = {2,7,1,2,5,5,2,9,3,2,8};
int maxCount = 0;
int element = 0;

for(int i=0;i<arr.length;i++)
{
int cnt = 0;
for(int j=0;j<arr.length;j++)
{
if(arr[i] == arr[j])
{
cnt++;
}
}
```

```
if(cnt>maxCount)
{
maxCount = cnt;
element = arr[i];
}
}
System.out.println(element+" repeating for "+maxCount+" times");
}
}
```

Q) Write a java program to display pair of array elements equals to sum 10?

input:
arr = 5 8 3 7 2 9 1
sum = 10
output:
8 2
3 7
9 1

```
class Test
{
public static void main(String[] args)
{
int[] arr = {5,8,3,7,2,9,1};
int sum = 10;

for(int i=0;i<arr.length;i++)
{
for(int j=i+1;j<arr.length;j++)
{
if(arr[i]+arr[j] == sum)
{
System.out.println(arr[i]+" "+arr[j]);
}
}
```

```
}
}
}
```

Q) Write a java program to display triple of array elements equals to sum 10?

input:
arr = 5 8 3 7 2 9 1
sum = 10
output:
5 3 2
7 2 1

```
class Test
{
public static void main(String[] args)
{
int[] arr = {5,8,3,7,2,9,1};
int sum = 10;

for(int i=0;i<arr.length;i++)
{
for(int j=i+1;j<arr.length;j++)
{
for(int k=j+1;k<arr.length;k++)
{
if(arr[i]+arr[j]+arr[k] == sum)
{
System.out.println(arr[i]+" "+arr[j]+" "+arr[k]);
}
}
}
}
}
}
```

Q) Write a java program to shift all 0's to right side?

input:
6 0 9 0 1 0 5

output:
6 9 1 5 0 0 0

```
class Test
{
public static void main(String[] args)
{
int[] arr = {6,0,9,0,1,0,5};

int pos = 0;
for(int i=0;i<arr.length;i++)
{
if(arr[i]!=0)
{
int temp = arr[i];
arr[i] = arr[pos];
arr[pos] = temp;
pos++;
}
}
//display elements
for(int n : arr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display array elements which are greater to its immediate predecessor and successor and consider first and last element having only one neighbour?

input:
7 2 5 32 4 12 90

output:
7 32 90

```
class Test
{
public static void main(String[] args)
{
int[] arr = {7,2,5,32,4,12,90};

if(arr[0]>arr[1])
System.out.print(arr[0]+" ");

for(int i=1;i<arr.length-1;i++)
{
if(arr[i]>arr[i-1] && arr[i]>arr[i+1])
{
System.out.print(arr[i]+" ");
}
}

if(arr[arr.length-1]>arr[arr.length-2])
{
System.out.print(arr[arr.length-1]);
}
}
}
```

Day:36_____ 05-09-26 Saturday
==========

Q) Write a java program to display distinct elements from given array?

input:

5 9 1 3 7 6 9 3 4 2 2

output:
5 9 1 3 7 6 4 2

```
class Test
{
public static void main(String[] args)f
{
int[] arr = {5,9,1,3,7,6,9,3,4,2,2};

for(int i=0;i<arr.length;i++)
{
int cnt = 0;
for(int j=0;j<i;j++)
{
if(arr[i] == arr[j])
{
cnt++;
break;
}
}
if(cnt==0)
System.out.print(arr[i]+" ");
}
}
}
```

Q) Write a java program to display all missing elements from given array?

input:
1 4 2 9

output:
3 5 6 7 8

```
import java.util.Arrays;
class Test
{
public static void main(String[] args)
{
int[] arr = {1,4,2,9};

Arrays.sort(arr);

for(int i=0;i<arr.length-1;i++)
{
int current = arr[i];
int next = arr[i+1];

while((current+1)<next)
{
System.out.print((current+1)+" ");
current++;
}
}



}
}
```

Q) Write a java program to insert element in a given index?

Input:
arr = 7 2 8 1 9 4
index = 3
insert = 100
Output:
7 2 8 100 1 9 4

```
import java.util.Arrays;
class Test
{
```

```
public static void main(String[] args)
{
int[] arr ={7,2,8,1,9,4};
int index = 3;
int insert = 100;

arr = Arrays.copyOf(arr,arr.length+1);

for(int i = arr.length-1;i>=index; i--)
{
arr[i] = arr[i-1];
}
arr[index] = insert;

for(int n : arr)
{
System.out.print(n+" ");
}
}
}
```

Q) Write a java program to display leader elements from given array?

Input:
2 8 72 9 34 6 12

Output:
72 34 12

```
import java.util.Arrays;
class Test
{
public static void main(String[] args)
{
int[] arr ={2,8,72,9,34,6,12};
```

```
for(int i=0;i<arr.length;i++)
{
boolean flag = true;
for(int j=i+1;j<arr.length;j++)
{
if(arr[i]>arr[j])
{
continue;
}
else
{
flag = false;
break;
}
}
if(flag==true)
System.out.print(arr[i]+" ");
}

}
}
```

Double Dimensional Array

==========

Double dimensional array is a combination of rows and columns.

Double dimensional array is implemented based on array of arrays approach but not matrix form.

The main objective of double dimensional array is memory utilization.

Double dimensional array is used to develop business oriented applications, gaming applications, matrix type of applications and etc.

We can declare and create double dimensional array as follow.

ex:

rows columns

| |

```
int[][] arr = new int[3][3];
```

Here we can store 9 elements.

We can initialize double dimentional array as follow.

ex:
int[][] arr = {{1,2,3},{4,5,6},{6,7,9}};

Programs
=========

Q) Write a java program to accept array elements and display them in matrix form?

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);

System.out.println("Enter the rows : ");
int rows = sc.nextInt();//3

System.out.println("Enter the columns : ");
int cols = sc.nextInt();//3

int[][] arr = new int[rows][cols];

//insert elements
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
System.out.println("Enter the array elements : ");
arr[i][j] = sc.nextInt();
}
}
```

```
//display elements
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
System.out.print(arr[i][j]+" ");
}
//new line
System.out.println();
}

}
}
```

Q) Write a java program to display array elements from double dimensional array?
input:
1 2 3
4 5 6
7 8 9

ex:

```
class Test
{
public static void main(String[] args)
{
int[][] arr = {{1,2,3},{4,5,6},{7,8,9}};

int rows = arr.length;
int cols = arr[0].length;

for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
```

```
System.out.print(arr[i][j]+" ");
}
//new line
System.out.println();
}
}
}
```

Approach2

```
class Test
{
public static void main(String[] args)
{
int[][] darr = {{1,2,3},{4,5,6},{7,8,9}};

for(int[] sarr : darr)
{
for(int n : sarr)
{
System.out.print(n+" ");
}
//new line
System.out.println();
}
}
}
```

Q) Write a java program to display sum of diagonal of a matrix?

input:
1 2 3
4 5 6
7 8 9

output:
15

```
class Test
{
public static void main(String[] args)
{
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

int rows = matrix.length;
int cols = matrix[0].length;

int sum = 0;
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
if(i==j)
{
sum += matrix[i][j];
}
}
}
System.out.println(sum);
}
}
```

Q) Write a java program to display sum of upper triangle elements?

input:
1 2 3
4 5 6
7 8 9

output:
11

```
class Test
{
public static void main(String[] args)
{
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

int rows = matrix.length;
int cols = matrix[0].length;

int sum = 0;
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
if(i<j)
{
sum += matrix[i][j];
}
}
}
System.out.println(sum);
}
}
```

Q) Write a java program to display sum of lower triangle elements?

input:
1 2 3
4 5 6
7 8 9

output:
19

```
class Test
{
public static void main(String[] args)
{
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

int rows = matrix.length;
int cols = matrix[0].length;

int sum = 0;
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
if(i>j)
{
sum += matrix[i][j];
}
}
}
System.out.println(sum);
}
}
```

Q) Write a java program to perform transpose of a matrix?

input:
1 2 3
4 5 6
7 8 9

output:
1 4 7
2 5 8
3 6 9

```
class Test
{
public static void main(String[] args)
{
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

int rows = matrix.length;
int cols = matrix[0].length;

int[][] transpose = new int[rows][cols];

for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
transpose[i][j] = matrix[j][i];
}
}

//display
for(int[] sarr : transpose)
{
for(int n : sarr)
{
System.out.print(n+" ");
}
//new line
System.out.println();
}

}
}
```

Q) Write a java program to sort the matrix?

input:

4 9 1
6 3 7
2 5 8

output:
1 2 3
4 5 6
7 8 9

```
import java.util.Arrays;
class Test
{
public static void main(String[] args)
{
int[][] matrix = {{4,9,1},{6,3,7},{2,5,8}};

int rows = matrix.length;
int cols = matrix[0].length;

int[] flatArr = new int[rows*cols];

//convert double dimensional array to single dimensional array
int index = 0;
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
{
flatArr[index++]= matrix[i][j];
}
}
Arrays.sort(flatArr);

//convert single dimensional array to double dimensional array
int idx = 0;
for(int i=0;i<rows;i++)
{
for(int j=0;j<cols;j++)
```

```
{
matrix[i][j] = flatArr[idx++];
System.out.print(matrix[i][j]+" ");
}
//new line
System.out.println();
}
}
}
```

Q) Write a java program to display array elements in spiral form?

input:
1 2 3
4 5 6
7 8 9
output:
1 2 3 6 9 8 7 4 5

```
class Test
{
public static void main(String[] args)
{
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

int top = 0;
int bottom = matrix.length-1;
int left = 0;
int right = matrix[0].length-1;

while(true)
{
if(left>right)
{
break;
}
```

```
for(int i=left;i<=right;i++)
{
System.out.print(matrix[top][i]+" ");
}
top++;

if(top>bottom)
{
break;
}
for(int i=top;i<=bottom;i++)
{
System.out.print(matrix[i][right]+" ");
}
right--;

if(left>right)
{
break;
}
for(int i=right;i>=left;i--)
{
System.out.print(matrix[bottom][i]+" ");
}
bottom--;
if(top>bottom)
{
break;
}
for(int i=bottom;i>=top;i--)
{
System.out.print(matrix[i][left]+" ");
}
left++;
}
}
}
```

Day:37_____ 07-09-26 Monday

==========

Multi-Dimensional Array

==========

If array contains more then two dimensions is called multi-dimensional array.

Multi dimensional array is used to display the data in structured and tabular format.

We can declare and create multi dimensional array as follow.

ex: base index

|

int[][][] arr = new int[2][3][3];

| |

rows columns

Here we can store 18 elements.

We can initialize multi-dimensional array as follow.

ex:

```
int[][][] arr = {
{{1,2,3},{4,5,6},{7,8,9}},
{{1,2,3},{4,5,6},{7,8,9}}
};
```

ex:

```
class Test
{
public static void main(String[] args)
{
int[][][] arr = {
{{1,2,3},{4,5,6},{7,8,9}},
{{1,2,3},{4,5,6},{7,8,9}}
};

for(int i=0;i<arr.length;i++)
```

```
{
for(int j=0;j<arr[i].length;j++)
{
for(int k=0;k<arr[i][j].length;k++)
{
System.out.print(arr[i][j][k]+" ");
}
System.out.println();
}
System.out.println("");
}

}
}
```

Jagged Array

==========

Jagged array is a also known as array of arrays.

It is a multi-dimensional array where each row contains different column size.

ex:

```
class Test
{
public static void main(String[] args)
{
int[][] arr = { {1,2},
{3,4,5},
{6,7,8,9}
};

for(int i=0;i<arr.length;i++)
{
for(int j=0;j<arr[i].length;j++)
{
System.out.print(arr[i][j]+" ");
}
//new line
System.out.println();
```

```
}
}
}
```

Anonymous Array

==========

Sometime we will declare an array without name such type of nameless array is called anonymous array.

The main objective of anonymous array is just for instance use.

We can declare anonymous array as follow.

ex:

```
new int[]{10,20,30}
new int[][]{{1,2,3},{4,5,6},{7,8,9}}
```

ex:

```
class Test
{
public static void main(String[] args)
{
System.out.println(sum(new int[]{10,20,30,40}));
}
public static int sum(int[] arr)
{
int sum = 0;
for(int n : arr)
{
sum+=n;
}
return sum;
}
}
```

OOPS

=====

OOPS stands for Object Oriented Programming System.

OOPS introduced to deal with realworld entities using programming language.

A language which allows us to declare classes and objects and supports OOPS principals(abstraction, encapsulation, inheritance and polymorphism) is called object oriented programming language.

A language is said to be object oriented if it is supports following features.

ex:

class

object

abstraction

encapsulation

inheritance

and

polymorphism

class

=======

A class is a blue print of an object.

ex:

design

template

It is a logical entity.

It is a collection of objects.

A class will accept following modifiers.

ex:

default

public

final

abstract

sealed

non-sealed

We can declare a class as follow.

syntax:
---
class class_name <extends> parent_class_name
<implements> interface_name
{
-
- // set of objects
-
}

object
=========
It is a outcome of a blue print.

It is a instance of a class. Here instance means allocating memory for our data members.

It is a physical entity.

It is a collection of properties and behaviours.
ex:
Dog
|
|---------------------------------|
Properties Behaviours

> Name > Eating
> Color > Sleeping
> Breed > playing
> Height > running
> Weight > barking
and etc. and etc.

We can create multiple objects in a single class.

We can create an object as follow.

ex:
reference variable
|
Test t = new Test();
| | |
classname keyword constructor

ex:
---
class Test
{
public static void main(String[] args)
{
Test t1 = new Test();
Test t2 = new Test();
System.out.println(t1.hashCode());//876213901
System.out.println(t2.hashCode());//230528013
System.out.println(t1); // Test@Hexadecimalno
System.out.println(t2.toString()); // Test@Hexadecimalno
}
}

Q) What is hashcode in java?

For every object, JVM creates a unique identification number i.e hash code.
In order to read hashcode of an object we need to use hashCode() method.
A hashCode() method present in java.lang.Object class.

Diagram: class29.1

Q) What is toString() method?

Whenever we are trying to display any object reference directly or indirectly toString() method will be executed.

A toString() method present in java.lang.Object class.

ex:

```
class Test
{
public static void main(String[] args)
{
int[] arr = new int[10];
System.out.println(arr); // [I@hexadecimalno
System.out.println(arr.toString()); // [I@hexadecimalno
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
Test t = new Test();
System.out.println(t); // Test@hexadecimalno
System.out.println(t.toString()); // Test@hexadecimalno
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
Test t = new Test();
System.out.println(t);
}
@Override
public String toString()
```

```
{
return "welcome to java";
}
}
```

Q) What is the difference between class and object?

class	object
It is a blue print of an object.	It is a instance of a class.
It is a logical entity.	It is a physical entity.
It is a collection of objects.	It is a collection of properties & behaviours.
To declare a class we will use class keyword.	To declare an object we will use new keyword.
It can't be redeclared.	It can be redeclared.
It can't be modified.	It can be modified.

Data Hiding
==========
The process of hiding object data from unauthorized access is called data hiding.

The main objective of data hiding is to provide security.

Using private modifier we can achieve data hiding concept.

ex:

```
class Account
{
private double balance = 10000d;
}
class Test
```

```
{
public static void main(String[] args)
{
Account acc = new Account();
System.out.println(acc.balance);
}
}
```

Abstraction

==========

Hiding internal implementation and highlighting the set of services is called abstraction.

Using abstract classes and interfaces we can implement abstraction.

Best example of abstraction is GUI ATM machine where bank people will hide internal implementation and highlights set of services like banking, withdrawal, mini statement and etc.

The main advantages of abstraction are

1) It gives security because it hides internal implementation.

2) Enhancement becomes more easy because without effecting end user they can perform any changes in our internal system.

3) It provides flexibility to the end user to use the system.

4) It improves maintainability of an application.

Encapsulation

==========

Encapsulation is related to capsule.

The process of encapsulating or wrapping variables and it's associate methods in a single entity is called encapsulation.

A class is said to be encapsulated class if it supports data hiding and abstraction.

Abstraction is used to hide the data and encapsulation is used to protect the data.

In encapsulation for every variable we need to declare setter and getter methods.

The main advantages of encapsulation are

1) It gives security
2) Enhancement becomes more easy.
3) It provides flexibility to the end user.
4) It improves maintainability of an application.

The main disadvantage of encapsulation is, it increases length of our code and slowdown the execution process.

ex:

```
class Student
{
//current class variables
private int studId;
private String studName;
private double studFee;

//setter methods
public void setStudId(int studId)
{
this.studId = studId;
}
public void setStudName(String studName)
{
this.studName = studName;
}
public void setStudFee(double studFee)
{
this.studFee = studFee;
}

//getter methods
```

```
public int getStudId()
{
return studId;
}
public String getStudName()
{
return studName;
}
public double getStudFee()
{
return studFee;
}
}
class Test
{
public static void main(String[] args)
{
Student s = new Student();
s.setStudId(101);
s.setStudName("Alan");
s.setStudFee(10000d);

System.out.println("Student Id : "+s.getStudId());
System.out.println("Student Name : "+s.getStudName());
System.out.println("Student Fee : "+s.getStudFee());
}
}
```

ex:

```
class Student
{
//current class variables
private int studId;
private String studName;
private double studFee;
```

```
//setter methods
public void setStudId(int studId)
{
this.studId = studId;
}
public void setStudName(String studName)
{
this.studName = studName;
}
public void setStudFee(double studFee)
{
this.studFee = studFee;
}

//getter methods
public int getStudId()
{
return studId;
}
public String getStudName()
{
return studName;
}
public double getStudFee()
{
return studFee;
}

//toString method
@Override
public String toString()
{
return studId+" "+studName+" "+studFee;
}
}
class Test
{
public static void main(String[] args)
```

```
{
Student s = new Student();
s.setStudId(101);
s.setStudName("Alan");
s.setStudFee(10000d);

System.out.println("Student Id : "+s.getStudId());
System.out.println("Student Name : "+s.getStudName());
System.out.println("Student Fee : "+s.getStudFee());

System.out.println(s);
}
}
```

Q) What is the difference between Abstraction and Encapsulation?

Abstraction	Encapsulation
Hiding internal implementation and highlighting set of services is called abstraction.	The process of encapsulating or wrapping variables and it's associate methods in a single entity is called encapsulation.
It is used to hide the data.	It is used to protect the data.
Using abstract classes and interface we can implements abstraction.	Using access modifiers we can implements encapsulation.
It is a process of gaining the information.	It is a process of containing the information.
It solves the issue at design level.	It solves the issue at implementation level.

Q) What is difference between POJO and Java Bean class?

POJO	Java Bean
It can't be serialized.	It can be serialized.

Fields can have any visibility. Fields must have private visibility.

It may or may not have constructor. It must have at least zero argument constructor.

It can't extend outside classes. It can extends outside class.

It can't implement outside interfaces. It can implements outside interfaces.

It can't use outside annotations. It can use outside annotations.

Day:38_____ 08-09-26 Tuesday

==========

Is-A relationship

==========

Is-A relationship is also known as inheritance.

Using extends keyword we can implements Is-A relationship.

The main objective of Is-A relationship is to provide reusability.

ex:
Animal is a dog and Dog is a animal
Vehicle is a car and car is a vehicle
Bird is a sparrow and sparrow is a bird.

ex:

```
class Animal
{
public void eat()
{
System.out.println("Eat Method");
}
}
class Dog extends Animal
{
```

```
public void sound()
{
System.out.println("Sound Method");
}
}
class Test
{
public static void main(String[] args)
{
Animal a = new Animal();
a.eat();

Dog d = new Dog();
d.eat();
d.sound();

Animal a1 = new Dog();
a.eat();

// Dog d1 = new Animal(); // C.T.E
}
}
```

conclusion

Whenever parent having properties by default go to child. But whatever child having properties by default it never go to parent.

A parent reference can hold child object but child reference can't hold parent object.

Inheritance

==========

It is a mechanism in which one class acquires the properties of another class.

It is a mechanism where new class derives the properties and behaviours from existing class.

Inheritance promotes code reusability.

Diagram: class30.1

We have following list of inheritance.

1) Single Level Inheritance

2) Multi-Level Inheritance

3) Multiple Inheritance

4) Hierarchical Inheritance

5) Hybrid Inheritance .

1) Single Level Inheritance

If we derived a class in the presence of one base class is called single level inheritance.

diag:

A (Parent/Base/Super class)

|

|

B (Child/Derived/Sub class)

ex:

```
class Father
{
public void skill1()
{
System.out.println("Coding Skill");
}
}
class Son extends Father
{
public void skill2()
{
```

```
System.out.println("Fast Riding Bike Skill");
}
}
class Test
{
public static void main(String[] args)
{
Father f = new Father();
f.skill1();

Son s = new Son();
s.skill1();
s.skill2();
}
}
```

2) Multi-Level Inheritance

```
class GrandFather
{
public void property1()
{
System.out.println("House");
}
}
class Father extends GrandFather
{
public void property2()
{
System.out.println("Car");
}
}
class Son extends Father
{
public void property3()
{
System.out.println("Bike");
```

```
}
}
class Test
{
public static void main(String[] args)
{
GrandFather gf = new GrandFather();
gf.property1();

Father f = new Father();
f.property1();
f.property2();

Son s = new Son();
s.property1();
s.property2();
s.property3();

}
}
```

3) Multiple Inheritance

In java, we can't extends more then one class simultaneously because java does not support multiple inheritance.

ex:

```
class A
{
}
class B
{
}
class C extends A,B --> invalid
{
}
```

In java, interface can extends more then one interface so we can achieve multiple inheritance concept through interfaces.

ex:

```
interface A
{
}
interface B
{
}
interface C extends A,B
{
}
```

If our class does not extends any other class then it is a direct child class of Object class.

ex: diag:

class A Object

{ |

} |

A

If our class extends some other class then it is a indirect child class of Object class.

ex: diag:

class A Object

{ |

} A

class B extends A |

{ B

}

Java does not support cyclic inheritance.

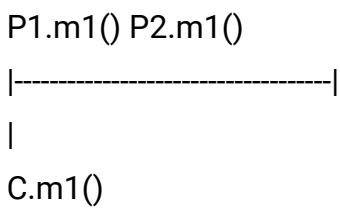
ex:

```
class A extends B
{
}
class B extends A
{
}
```

Q) Why java does not support multiple inheritance?

There is a chance of raising ambiguity problem that's why java does not support multiple inheritance.

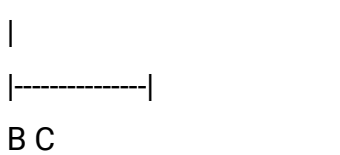
ex:



4) Hierarchical Inheritance

If we derived multiple classes by using one base class is called hierarchical inheritance.

diag:



ex:

```
class Animal
{
public void eat()
{
System.out.println("Animal Eat");
}
}
class Dog extends Animal
{
public void sound()
{
System.out.println("Bow Bow");
}
}
```

```
class Cat extends Animal
{
public void sound()
{
System.out.println("Meow Meow");
}
}
class Test
{
public static void main(String[] args)
{
Animal a = new Animal();
a.eat();

Dog d = new Dog();
d.eat();
d.sound();

Cat c = new Cat();
c.eat();
c.sound();
}
}
```

5) Hybrid Inheritance

It is a combination of two or more inheritance.

Java does not support hybrid inheritance.

diag:

```
A
|
|----------|
B C
|----------|
|
D
```

Constructor

==========

It is a special method which is used to initialize an object.

Having same name as class name is called constructor.

In java, constructor will execute when we create instance of a class.

It does not allow any return type. If we declare any return type then we won't get any compile time error or runtime error.

A constructor will accept following modifiers.

ex:

default

public

private

protected

In java , constructors are divided into two types.

1) Userdefined constructor

2) Default constructor

1) Userdefined constructor

A constructor which is created by the user based on the application requirements is called userdefined constructor.

It is categorized into two types.

i) Zero Argument Constructor

ii) Parameterized Constructor

i) Zero Argument Constructor

Suppose we are not passing any argument to userdefined constructor then such concept is called zero argument constructor.

ex:

```
class Test
{
Test()
{
System.out.println("0-arg const");
}
public static void main(String[] args)
{
System.out.println("main-method");
}
}
```

o/p:

main-method

ex:

```
class Test
{
public Test()
{
System.out.println("0-arg const");
}
public static void main(String[] args)
{
System.out.println("main-method");
Test t = new Test();
}
}
```

o/p:

main-method

0-arg const

ex:

```
class Test
{
private Test()
{
System.out.println("0-arg const");
}
public static void main(String[] args)
{
Test t1 = new Test();
System.out.println("main-method");
Test t2 = new Test();
}
}
```

o/p:

```
0-arg const
main-method
0-arg const
```

ex:

```
class Test
{
protected Test()
{
System.out.println("0-arg const");
}
public static void main(String[] args)
{
Test t1 = new Test();
System.out.println("main-method");
Test t2 = new Test();
}
}
```

o/p:

0-arg const

main-method

0-arg const

ii) Parameterized Constructor

Suppose if we pass atleat one argument to userdefined constructor such type of constructor is called parameterized constructor.

ex:

```
class Employee
{
//current class variables
private int empId;
private String empName;
private double empSal;

Employee(int empId,String empName,double empSal)
{
this.empId = empId;
this.empName = empName;
this.empSal = empSal;
}
public void getEmployeeDetails()
{
System.out.println("Employee Id : "+empId);
System.out.println("Employee Name : "+empName);
System.out.println("Employee Salary : "+empSal);
}
}
class Test
{
public static void main(String[] args)
{
Employee e = new Employee(101,"Alan",10000d);
e.getEmployeeDetails();
}
```

}

2) Default constructor

It is a compiler generated constructor for every java program where we are not defining atleast zero argument constructor.

To see default constructor we need to use below command.

ex:

javap -c Test

Diagram: class30.2

Constructor Overloading

==========

Having same constructor name with different parameters in a single class is called constructor overloading.

ex:

```
class A
{
A()
{
System.out.println("0-arg const");
}
A(int i)
{
System.out.println("int-arg const");
}
A(double d)
{
System.out.println("double-arg const");
}
}
class Test
{
```

```
public static void main(String[] args)
{
A a1 = new A();
A a2 = new A(10);
A a3 = new A(10.5d);
}
}
```

Day:39_____ 10-09-26 Thursday

==========

Has-A Relationship

==========

Has-A relationship is also known as composition and aggregation.

There is no specific key word to implements Has-A relationship but mostly we will use new keyword.

The main objective of Has-A relationship is to provide reusability.

Has-A relationship increases dependency between two components.

ex:

A course has a content

A car has a engine

A laptop has a charger

and etc.

ex:

```
class Content
{
public String java()
{
return "React+SpringBoot+OracleDB";
}
public String dataScience()
```

```
{
return "Python+ML+DL";
}
}
class Course
{
public void courseDetails()
{
Content c = new Content();
System.out.println("Java Content : "+c.java());
System.out.println("Data Science Content :"+c.dataScience());
}
}
class RaiseTech
{
public static void main(String[] args)
{
Course c = new Course();
c.courseDetails();
}
}
```

composition

==========

Without existing container object there is no chance of having contained object then the relationship between container and contained object is called composition which is strongly association.

Diagram: class31.1

ex:

--

```
class Engine
{
public void engineStart()
{
System.out.println("Engine Started");
}
}
```

```
class Car
{
Engine e;
Car()
{
e = new Engine();
}
public void carStart()
{
e.engineStart();
System.out.println("Car Moving");
}
}
class Test
{
public static void main(String[] args)
{
Car c = new Car();
c.carStart();
}
}
```

Aggregation

==========

Without existing container object there is a chance of having contained object then the relationship between container and contained object is called aggregation which is loosely association.

Diagram: class31.2

ex:

```
class Engine
{
public void engineStart()
{
System.out.println("Engine Started");
}
```

```
}
class Car
{
Engine e;
Car(Engine e)
{
this.e = e;
}
public void carStart()
{
e.engineStart();
System.out.println("Car Moving");
}
}
class Test
{
public static void main(String[] args)
{
Engine e = new Engine();
Car c = new Car(e);
c.carStart();
}
}
```

Method overloading

==========

Having same method name with different signatures in a single class is called method overloading.

All the methods which are present in a class are called overloaded methods.

Method resolution will taken care by a compiler based on reference type.

Method overloading reduces complexity of the programming.

ex:

--

```
class ECI
```

```
{
//overloaded methods
public void search(int epicNo)
{
System.out.println("Details Found via Epic No");
}
public void search(long phoneNo)
{
System.out.println("Details Found via Phone No");
}
public void search(String details)
{
System.out.println("Details Found via House No");
}
}
class Voter
{
public static void main(String[] args)
{
ECI eci = new ECI();
eci.search(34567);
eci.search(9876543287L);
eci.search("1-7-453/1");
}
}
```

Q) Can we overload main method in jaa?

Yes, we can overload main method in java but JVM always execute main method with String[] parameter only.

ex:

```
class Test
{
public static void main(int[] iargs)
```

```
{
System.out.println("int[] iargs");
}
public static void main(String[] args)
{
System.out.println("String[] args");
}
}
```

Method Overriding
==========
Having same method name with same parameters in two different classes is called method overriding.

Methods which are present in parent class are called overridden methods.

Methods which are present in child class are called overriding methods.

Method resolution will taken care by a JVM based on runtime object.

ex:

```
class Parent
{
//overridden methods
public void property()
{
System.out.println("Cash+Gold+Land");
}
public void marry()
{
System.out.println("Trisha");
}
}
class Child extends Parent
{
//overridding methods
```

```
@Override
public void marry()
{
System.out.println("Rashmika");
}
}
class Test
{
public static void main(String[] args)
{
Parent p = new Parent();
p.property(); // Cash+Gold+Land
p.marry(); // Trisha

Child c = new Child();
c.property(); // Cash+Gold+Land
c.marry(); // Rashmika

Parent p1 = new Child();
p1.property(); // Cash+Gold+Land
p1.marry(); // Rashmika
}
}
```

If we declare any method as final then overridding of that method is not possible.

ex:

```
class Parent
{
public final void property()
{
System.out.println("House-Not For Sale");
}
}
class Child extends Parent
{
```

```
@Override
public void property()
{
System.out.println("House-For Sale");
}
}
class Test
{
public static void main(String[] args)
{
Parent p = new Child();
p.property();
}
}
```

o/p:

C.T.E : Overridden method is final

Method Hiding

==========

Method hiding is exactly same as method overriding with following differences.

Method Overriding	Method Hiding
Methods should be non-static.	Methods should be static.
Method resolution will taken care by JVM based on runtime object.	Method resolution will taken care by compiler based on reference type.

It is also known as runtime polymorphism, dynamic polymorphism or late binding.

It is also known a compile time polymorphism, static polymorphism or early binding.

ex:

```
class Parent
{
public static void property()
```

```
{
System.out.println("House-Not For Sale");
}
}
class Child extends Parent
{
public static void property()
{
System.out.println("House-For Sale");
}
}
class Test
{
public static void main(String[] args)
{
Parent p = new Child();
p.property(); // House- Not For Sale
}
}
```

Q) Can we override main method in java?

No, we can't override main method in java because it is static.

Polymorphism

==========

Polymorphism has derived from two Greek words.

Here poly means many and morphism means forms.

The ability to represent in different forms is called polymorphism.

Diagram: class31.3

The main objective of polymorphism is to provide flexibility.

In java, polymorphism is divided into two types.

1) Compile time polymorphism

2) Runtime polymorphism

1) Compile time polymorphism

A polymorphism which exhibits at compile time is called compile time polymorphism.
Methods resolution will taken care by a compiler based on reference type.
It is also known as static polymorphism or early binding.
ex:
Method Overloading
Method Hiding

2) Runtime polymorphism

A polymorphism which exhibits at runtime time is called runtime polymorphism.
Methods resolution will taken care by a JVM based on runtime object.
It is also known as dynamic polymorphism or late binding.
ex:
Method Overriding

this keyword
==========
A this keyword is a java keyword which is used to refer current class object reference.

We can utilize this keyword in following ways.

1) To refer current class variables

2) To refer current class methods

3) To refer current class constructors

1) To refer current class variables

```
class A
{
```

```
int i = 10;
int j = 20;
A(int i,int j)
{
System.out.println(i+" "+j); // 100 200
System.out.println(this.i+" "+this.j);// 10 20
}
}
class Test
{
public static void main(String[] args)
{
A a = new A(100,200);
}
}
```

2) To refer current class methods

```
class A
{
public void m1()
{
System.out.println("M1-Method");
this.m2();
}
public void m2()
{
System.out.println("M2-Method");
}
}
class Test
{
public static void main(String[] args)
{
A a = new A();
a.m1();
}
}
```

3) To refer current class constructors

```
class A
{
A()
{
System.out.println("0-arg const");
}
A(int i)
{
this();
System.out.println("int-arg const");
}
A(double d)
{
this(10);
System.out.println("double-arg const");
}
}
class Test
{
public static void main(String[] args)
{
A a1 = new A(10.5d);
}
}
```

super keyword

==========

A super is a java keyword which is used to refer super class object reference.

We can utilize super keyword in following ways.

1) To refer super class variables

2) To refer super class methods

3) To refer super class constructors

1) To refer super class variables

```
class A
{
int i=1;
int j=2;
}
class B extends A
{
int i=10;
int j=20;
B(int i,int j)
{
System.out.println(this.i+" "+this.j); // 10 20
System.out.println(i+" "+j); // 100 200
System.out.println(super.i+" "+super.j);// 1 2
}
}
class Test
{
public static void main(String[] args)
{
B b = new B(100,200);
}
}
```

2) To refer super class methods

```
class A
{
public void m1()
{
System.out.println("M1-Method");
}
}
class B extends A
```

```
{
public void m2()
{
super.m1();
System.out.println("M2-Method");
}
}
class Test
{
public static void main(String[] args)
{
B b = new B();
b.m2();
}
}
```

3) To refer super class constructors

```
class A
{
A()
{
System.out.println("A-const");
}
}
class B extends A
{
B()
{
//super();
System.out.println("B-const");
}
}
class Test
{
public static void main(String[] args)
{
B b = new B();
```

}
}

Day:40_____ 11-09-26 Friday

==========

Interfaces

==========

Interface is a blue print of a class and it is a collection of abstract methods, default methods, static methods and private methods.

Diagram: class32.1

Abstract method is a incomplete method because it ends with semicolon and does not have any body.

ex:

public abstract void methodOne();

It is not possible to create object for interfaces because it contains unimplemented methods and lack of constructor.

To write the implementation of abstract methods we will use implementation class.

It is possible to create object for implementation class because it contains method with body.

By default every abstract method is a public and abstract.

Interface contains only constants i.e public static final.

If we have service requirement specifications then we need to use interface.

ex:

---

interface Animal

{

public abstract void sound();

```
}
class Dog implements Animal
{
@Override
public void sound()
{
System.out.println("Bow Bow");
}
}
class Test
{
public static void main(String[] args)
{
Animal a = new Dog();
a.sound();
}
}
```

ex:

```
interface Animal
{
public abstract void sound();
}
class Test
{
public static void main(String[] args)
{
//Anonymous inner class
Animal a = new Animal()
{
public void sound()
{
System.out.println("Meow Meow");
}
};
a.sound();
}
```

}

If interface contains four methods then we need to override all methods otherwise we will get compile time error.

ex:

```
interface ATM
{
public abstract void deposit();
public void withdrawl();
abstract void ministatement();
void transfer();
}
class ATMImpl implements ATM
{
@Override
public void deposit()
{
System.out.println("Deposit Method");
}
@Override
public void withdrawl()
{
System.out.println("Withdrawl Method");
}
@Override
public void ministatement()
{
System.out.println("MiniStatement Method");
}
@Override
public void transfer()
{
System.out.println("Transfer Method");
}
}
class Test
```

```
{
public static void main(String[] args)
{
ATM atm = new ATMImpl();
atm.deposit();
atm.withdrawl();
atm.ministatement();
atm.transfer();
}
}
```

In java, class can't extends more then one class.
But interface can extends more then one interface.

ex:

```
interface A
{
void methodOne();
}
interface B
{
void methodTwo();
}
interface C extends A,B
{
void methodThree();
}
class D implements C
{
@Override
public void methodOne()
{
System.out.println("MethodOne");
}
@Override
public void methodTwo()
{
```

```
System.out.println("MethodTwo");
}
@Override
public void methodThree()
{
System.out.println("MethodThree");
}
}
class Test
{
public static void main(String[] args)
{
C c = new D();
c.methodOne();
c.methodTwo();
c.methodThree();
}
}
```

A class can implements more then one interface.

ex:

```
interface Father
{
float HT = 6.2f;
void height();
}
interface Mother
{
float HT = 5.8f;
void height();
}
class Child implements Father,Mother
{
@Override
public void height()
{
```

```
float height = (Father.HT+Mother.HT)/2;
System.out.println("Child Height ="+height);
}
}
class Test
{
public static void main(String[] args)
{
Child c = new Child();
c.height();
}
}
```

Marker interface

==========

Interface which does not contain any methods and constants is called marker interface.

In general, empty interface is called marker interface.

We have following list of marker interfaces in java.

ex:

Serializable

Cloneable

Remote

and etc.

Using marker interface we will get some ability to do.

Q) Write a program to achieve multiple inheritance in java?

```
interface Father
{
public void skill();
}
interface Mother
```

```
{
public void skill();
}
class Child implements Father,Mother
{
@Override
public void skill()
{
System.out.println("Programming Skill");
System.out.println("Cooking Skill");
}
}
class Test
{
public static void main(String[] args)
{
Child c = new Child();
c.skill();
}
}
```

Abstract class
==========
It is a collection of zero or more abstract methods or concrete(regular) methods.

A abstract keyword is applicable for methods and classes but not for variables.

It is not possible to create object for abstract class because it contains incomplete methods.

To write the implementation of abstract methods we will use sub classes.

In abstract class, abstract methods must be declare using abstract keyword.
ex:
abstract void m1();
public abstract void m1();

Abstract class contains only instance variables.

If we know partial implementation the we need to use abstract class.

ex:

```
abstract class Payment
{
public abstract void paymentMethod();
}
class CardPayment extends Payment
{
@Override
public void paymentMethod()
{
System.out.println("Card Payment");
}
}
class Test
{
public static void main(String[] args)
{
CardPayment cp = new CardPayment();
cp.paymentMethod();
}
}
```

ex:

```
abstract class Plan
{
//instance variable
protected double rate;

//abstract method
public abstract void getRate();

//concret method
public void calculateBillAmt(int units)
{
```

```
System.out.println("Total Units : "+units);
System.out.println("Total Bill : "+ (rate*units));
}
}
class DomesticPlan extends Plan
{
@Override
public void getRate()
{
rate = 2.5d;
}
}
class CommercialPlan extends Plan
{
@Override
public void getRate()
{
rate = 5.0d;
}
}
class Test
{
public static void main(String[] args)
{
DomesticPlan dp = new DomesticPlan();
dp.getRate();
dp.calculateBillAmt(250);

CommercialPlan cp = new CommercialPlan();
cp.getRate();
cp.calculateBillAmt(250);
}
}
```

Q) What is the difference between Interface and Abstract class?

Interface	Abstract class
To declare interface we will use interface keyword.	To declare abstract class we will use abstract keyword.
It is a blue print of a class and it is a collection of abstract methods,default methods, static methods and private methods.	It is a collection of zero or more abstract methods and concrete/regular methods.
To write implementation of abstract methods we will use implementation class.	To write implement of abstract methods we will use sub classes.
It does not allow constructor.	It allows constructor.
It does not allow blocks.	It allows blocks.
We can achieve multiple inheritance.	We can't achieve multiple inheritance.
If we know only specification then we need to use interface.	If we know partial implementation then we need to use abstract class.

Abstraction Example
==========

ex:1

```
abstract class Draw
{
public abstract void shape();
}
class Circle extends Draw
{
@Override
public void shape()
{
System.out.println("Circle Shape");
}
```

```
}
class Test
{
public static void main(String[] args)
{
Draw d = new Circle();
d.shape();
}
}
```

ex:2

```
interface Draw
{
public abstract void shape();
}
class Circle implements Draw
{
@Override
public void shape()
{
System.out.println("Circle Shape");
}
}
class Test
{
public static void main(String[] args)
{
Draw d = new Circle();
d.shape();
}
}
```

Day:41_____ 12-09-26 Saturday

==========

Packages

========

A package is a collection of classes, interfaces, enums and annotations.

Enum is a special class and Annotation is a special interface.

A package is also known as folder or a directory.

In java, packages are divided into two types.

1) Predefined packages

2) Userdefined packages

1) Predefined packages

Built-in packages are called predefined packages.

ex:

java.lang

java.io

java.util

java.time

java.util.stream

java.sql

javax.servlet

and etc

2) Userdefined packages

Packages which are created by the user based on the application requirements are called userdefined packages.

To declare userdefined package we need to use "package" keyword.

syntax:

---

package <package_name>;

It is always recommanded to declare package in reverse order of url.

ex:

```
package com.rt.www;
import java.util.Calendar;
class Test
{
public static void main(String[] args)
{
Calendar c = Calendar.getInstance();
int h = c.get(Calendar.HOUR_OF_DAY);
if(h<12)
System.out.println("Good Morning");
else if(h<16)
System.out.println("Good Afternoon");
else if(h<20)
System.out.println("Good Evening");
else
System.out.println("Good Night");
}
}
```

We can compile above program as follow.

ex:

current directory

|

javac -d . Test.java

|

destination folder

We can execute above program as follow.

ex:

classname

|

java com.rt.www.Test

|

packagename

Enum

=====

Enum concept introduced in Java 1.5 version.

Enum is a group of named constants.

Using enum we can create our own datatype called enumerated datatype.

When compare to old language enum, java enum is more powerful.

We can declare enum as follow.

syntax:

------

enum type_name

{

value1,value2,...,valueN

}

ex:

enum Months

{

JAN,FEB,MAR

}

Internal implementation of enum

Every enum internally consider as class concept and it extends with java.lang.Enum class.

Every enum constant is a reference variable of enum type.

ex:

enum Months final class Months extends java.lang.Enum

{ {

JAN,FEB,MAR => public static final Months JAN = new Months();

} public static final Months FEB = new Months();

public static final Months MAR = new Months();

```
}
```

Declaration and Usage of enum

```
enum Months
{
JAN,FEB,MAR
}
class Test
{
public static void main(String[] args)
{
Months m = Months.JAN;
System.out.println(m);
}
}
```

From Java 1.5 version onwards we can pass enum to switch case.

ex:

--

```
enum Drinks
{
COLA,COKE,PEPSI
}
class Test
{
public static void main(String[] args)
{
Drinks d = Drinks.COKE;
switch(d)
{
case COLA : System.out.println("Coca Cola"); break;

case COKE : System.out.println("Drink Coke Don't POKE"); break;

case PEPSI : System.out.println("Drink Pepsi future let's see");
```

```
}
}
}
```

java.lang.Enum

The power to enum will be inherited from java.lang.Enum class.

It contains following two methods.

1) values()

It returns group of constant from enum.

2) ordinal()

It returns ordinal number.

ex:

```
enum Week
{
MON,TUE,WED,THU,FRI,SAT,SUN
}
class Test
{
public static void main(String[] args)
{
Week[] w = Week.values();
for(Week w1 : w)
{
System.out.println(w1.ordinal()+"----------"+w1);
}
}
}
```

When compare to old language enum, java enum is more powerful because in addition to constants we can declare vaiables, methods and constructors.

ex:

```
enum Cloths
{
SILK,COTTON,KHADI;

Cloths()
{
System.out.println("constructor");
}
}
class Test
{
public static void main(String[] args)
{
Cloths c = Cloths.SILK;
}
}
```
o/p:
```
constructor
constructor
constructor
```

ex:
--
```
enum Programming
{
JAVA,PYTHON,PERL;

static int i = 10;

public static void main(String[] args)
{
System.out.println(i);
}
}
```

Wrapper classes

==========

The main objective of wrapper classes are.

1) To wrap primitive type to wrapper object and vice versa.

2) To defined several utility methods.

ex:

Primitive Type	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

constructor

Every wrapper class contains two constructors. One will take corresponding primitive as an argument and another will take corresponding String as an argument.

ex:

Wrapper class	Constructor
Byte	byte or String
Short	short or String
Integer	int or String
Long	long or String
Float	float or String
Double	double or String
Boolean	boolean or String
Character	char

ex:

---

class Test

```
{
public static void main(String[] args)
{
Integer i1 = Integer.valueOf(10);
System.out.println(i1);

Integer i2 = Integer.valueOf("20");
System.out.println(i2);
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
Boolean b1 = Boolean.valueOf(true);
System.out.println(b1);

Boolean b2 = Boolean.valueOf("false");
System.out.println(b2);
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
Character c = Character.valueOf('a');
System.out.println(c);
}
}
```

Utility Methods

We have following list of utility methods.

1) valueOf()

It is similar to constructor.

It is used to convert primitive type or String type to wrapper object.

ex:

```
class Test
{
public static void main(String[] args)
{
//primitive type
int a = 10;

//wrapper object
Integer i = Integer.valueOf(a);

System.out.println(i);
}
}
```

ex:

```
class Test
{
public static void main(String[] args)
{
//string type
String s = "200";

//wrapper object
Integer i = Integer.valueOf(s);

System.out.println(i);
}
}
```

2) xxxValue()

It is used to convert wrapper object to primitive type.

ex:

```
class Test
{
public static void main(String[] args)
{
//wrapper object
Integer i = Integer.valueOf(10);

//primitive type
byte b = i.byteValue();
System.out.println(b);

short s = i.shortValue();
System.out.println(s);

int a = i.intValue();
System.out.println(a);

long l = i.longValue();
System.out.println(l);
}
}
```

3) parseXxx()

It is used to convert string type to primitive or wrapper object.

ex:

```
class Test
{
```

```
public static void main(String[] args)
{
//string type
String s = "20";

//primitive type
int i = Integer.parseInt(s);
System.out.println(i);
}
}
```

4) toString()

It is used to convert wrapper object or primitive type to String.

ex:

--

```
class Test
{
public static void main(String[] args)
{
//wrapper object
Integer i = Integer.valueOf(10);

//string type
String s = i.toString();

System.out.println(s);
}
}
```

ex:

--

```
class Test
{
public static void main(String[] args)
```

```
{
//primitve type
int a = 100;

//string type
String s = Integer.toString(a);

System.out.println(s);
}
}
```

Interview Questions
==========

Q) Write a java program to perform sum of two binary numbers?

Input:
1010
0101
Output:
1111

```
import java.util.Scanner;
class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);

System.out.println("Enter the first binary number : ");
String binary1 = sc.next();

System.out.println("Enter the second binary number : ");
String binary2 = sc.next();

//convert binary to decimal
int a = Integer.parseInt(binary1,2);
int b = Integer.parseInt(binary2,2);
```

```
int c = a + b;

//convert decimal to binary number
String result = Integer.toBinaryString(c);
System.out.println(result);
}
}
```

Q) What is Autoboxing and Autounboxing in java?

Autoboxing

Automatic conversion from primitive type to corresponding wrapper object performed by the compiler is called autoboxing.Autoboxing internally uses valueOf() method.

ex:

```
class Test
{
public static void main(String[] args)
{
// primitive type
int a = 500;

//wrapper object
Integer i = a;
// Integer i = Integer.valueOf(a);


System.out.println(i);
}
}
```

Autounboxing

Automatic conversion from wrapper type to corresponding primitive type performed by the

compiler is called autounboxing.Autounboxing internally uses xxxValue() method.

ex:

```
class Test
{
public static void main(String[] args)
{
// wrapper object
Integer i = Integer.valueOf(10);

// primitve type
int a = i;
// int a = i.intValue();


System.out.println(a);
}
}
```

Q) Write a java program to convert number to string?

```
class Test
{
public static void main(String[] args)
{
int n = 10;
String s = Integer.toString(n);
System.out.println(s);
}
}
```

Q) Write a java program to convert String to number?

```
class Test
{
public static void main(String[] args)
{
String str= "60";
int n = Integer.parseInt(str);
System.out.println(n);
}
}
```

Q) Write a java program to convert character to number?

```
class Test
{
public static void main(String[] args)
{
char ch = '5';

int n = Character.getNumericValue(ch);

System.out.println(n);
}
}
```

Day:42_____ 15-09-26 Tuesday

==========

Inner classes

==========

Sometimes we will declare a class inside another class such concept is called inner class.

syntax:

```
class Outer
{
class Inner
{
```

- //code to be execute

}

}

Inner classes introduced in Java 1.1 version as a part of event handling to remove GUI bugs.

But due to powerful features and benefits of inner classes. Programmers started to use inner classes in regular programming.

Inner class does not support static members.

Inner class must and should declare with the scope of enclosing class.

We have four types of inner classes in java.

1) Normal/Regular inner class

2) Static inner class

3) Method local inner class

4) Anonymous inner class

1) Normal/Regular inner class

Sometimes we will declare a class inside another class without static keyword is called normal/ regular inner class.

ex:

```
class Outer
{
class Inner
{
public void methodOne()
{
System.out.println("Method One");
}
```

```
}
public static void main(String[] args)
{
Outer.Inner i = new Outer().new Inner();
i.methodOne();

//or
new Outer().new Inner().methodOne();
}
}
```

Note:

If we compile above program we will get two .class files i.e

Outer.class and Outer$Inner.class.

2) Static inner class

Sometimes we will declare a class inside another class using static keyword is called static inner class.

In regular/normal inner class, without existing container object there is no chance of having contained object.

ex:

```
class Outer
{
static class Inner
{
public void methodOne()
{
System.out.println("Method One");
}
}
public static void main(String[] args)
{
Inner i = new Inner();
i.methodOne();
```

```
}
}
```

3) Method local inner class

Method local inner class is used to execute method specific repeated execution.

ex:

```
class Outer
{
public void m1()
{
class Inner
{
public void sum(int a,int b)
{
System.out.println(a+b);
}
}
Inner i = new Inner();
i.sum(1,2);
i.sum(10,20);
i.sum(100,200);
}
public static void main(String[] args)
{
Outer o = new Outer();
o.m1();
}
}
```

4) Anonymous inner class

Sometimes we will declare a class without name such type of nameless class is called anonymous inner class.

ex:

```
interface ATM
{
public abstract void deposit();
}
class Test
{
public static void main(String[] args)
{
//anonymous inner class
ATM atm = new ATM()
{
public void deposit()
{
System.out.println("Deposit Method");
}
};
atm.deposit();
}
}
```

ex:

```
abstract class Payment
{
public abstract void paymentMethod();
}
class Test
{
public static void main(String[] args)
{
Payment p = new Payment()
{
public void paymentMethod()
{
System.out.println("Credit Card");
}
};
p.paymentMethod();
```

```
}
}
```

Cloning
=======
The process of creating exact duplicate object is called cloning.

We can perform cloning only for cloneable objects.

To create cloneable object our class must implements Cloneable interface.

Cloneable is a marker interface which does not have any methods and constants.

By using Cloneable interface we will get some ability to do by JVM.

To perform cloning we need to use clone() method of Object class.

ex:
```
protected native Object clone() throws CloneNotSupportedException
```

ex:

```
class Test implements Cloneable
{
//instance variable
int i = 10;

public static void main(String[] args)throws CloneNotSupportedException
{
Test t1 = new Test();
Test t2 = (Test)t1.clone();

System.out.println(t1.i);
System.out.println(t2.i);
```

```
System.out.println(t1.hashCode()); //564742142
System.out.println(t2.hashCode()); //90205195
}
}
```

Q) What is difference between shallow cloning and deep cloning?

Shallow Cloning

The process of creating exact duplicate object reference is called shallow cloning.

ex:

```
class Test
{
//instance variable
int i = 10;

public static void main(String[] args)
{
Test t1 = new Test();

Test t2 = t1;

System.out.println(t1.i); // 10
System.out.println(t2.i); // 10

System.out.println(t1.hashCode());//673186785
System.out.println(t2.hashCode());//673186785
}
}
```

Diagram: class34.1

Deep Cloning

The process of creating exact duplicate object is called deep cloning.

ex:

```
class Test implements Cloneable
{
//instance variable
int i = 10;

public static void main(String[] args)throws CloneNotSupportedException
{
Test t1 = new Test();

Test t2 = (Test)t1.clone();

System.out.println(t1.i); // 10
System.out.println(t2.i); // 10

System.out.println(t1.hashCode());//564742142
System.out.println(t2.hashCode());//90205195
}
}
```

Types of objects

==========

We have two types of objects in java.

1) Immutable object

2) Mutable object

1) Immutable object

After object creation , if we perform any changes then for every change new object will be created such type of object is called immutable object.

ex:

String and wrapper classes

In real life we will use immutable object when we won't allow to change the value of an object.

ex:

Date of birth, PAN number, account number, aadhar No and etc.

2) Mutable object

After object creation, if we perform any changes then all the required changes will be done in a same object such type of object is called mutable object.

ex:

StringBuffer and StringBuilder

In real life we will use mutable object when we allow to change the value of an object.

ex:

Salary , bank balance, shopping cart and etc.

String

======

It is a collection of characters enclosed in a double quotation.

case1:

Once if we create a String object we can't perform any changes. If we perform any changes then for every change a new object will be created such behaviour is called immutability of an object.

Diagram: class34.3

case2:

What is the difference between == and .equals() method?

==

It is used for reference or address comparision.

ex:

--

```
class Test
{

public static void main(String[] args)
{
String s1 = new String("raise");
String s2 = new String("raise");
System.out.println(s1 == s2); // false
}
}
```

.equals()

It is used for content comparision which is case sensitive.

ex:

```
class Test
{

public static void main(String[] args)
{
String s1 = new String("raise");
String s2 = new String("raise");
System.out.println(s1.equals(s2)); // true
}
}
```

case3:

Once if we create a String object. Two objects will be created. One is on heap and another is on SCP(String Constant Pool) area. But 's' always points to heap area.

Diagram: class34.4

Object creations in SCP area is always optional.If JVM will check is there any object is created with same content or not. If it is created then JVM simply refers to that object. If it is not created then JVM will create a new object. Hence there is no chance of having duplicate objects in SCP area.

Even though SCP objects do not have any reference. Garbage collector can't access them.

SCP objects destroy automatically whenever JVM shutdowns or terminated.

Diagram: class34.5

String important methods

==========

Q) Write a java program to display length of the string?

input:

raisetech

output:

9

```
class Test
{

public static void main(String[] args)
{
String str = "raisetech";
int len = str.length();
System.out.println(len);
}
}
```

Q) Write a java program to convert string to upper case?

input:

raisetech
output:
RAISETECH

```
class Test
{

public static void main(String[] args)
{
String str = "raisetech";
str = str.toUpperCase();
System.out.println(str);
}
}
```

Q) Write a java program to convert string to lower case?

input:
RAISETECH
output:
raisetech

```
class Test
{
public static void main(String[] args)
{
String str = "RAISETECH";
str = str.toLowerCase();
System.out.println(str);
}
}
```

Q) Write a java program to concatinate two strings?

input:
raise
tech
output:

raisetech

```
class Test
{
public static void main(String[] args)
{
String s1 = "raise";
String s2 = "tech";
String result = s1.concat(s2);
System.out.println(result);
}
}
```

Q) Write a java program to compare two strings?

input:
raisetech
raisetech
output:
Both are equals

```
class Test
{
public static void main(String[] args)
{
String s1 = "raisetech";
String s2 = "raisetech";
if(s1.equals(s2))
System.out.println("Both are same");
else
System.out.println("Both are not same");
}
}
```

Q) Write a java program to compare two strings?

input:
raisetech

RAISETECH
output:
Both are equals

```
class Test
{
public static void main(String[] args)
{
String s1 = "raisetech";
String s2 = "RAISETECH";
if(s1.equalsIgnoreCase(s2))
System.out.println("Both are same");
else
System.out.println("Both are not same");
}
}
```

Q) Write a java program to check string contains only alphabets?

input:
raisetech

output:
true

```
class Test
{
public static void main(String[] args)
{
String s = "raisetech";
System.out.println(s.matches("^[A-Za-z]+"));
}
}
```

Q) Write a java program to check string contains only digits?

input:
1234

output:
true

```
class Test
{
public static void main(String[] args)
{
String s = "1234";
System.out.println(s.matches("\\d+"));
}
}
```

Q) Write a java program to trim the characters from string?

input:
" raise tech "
output:
raise tech

```
class Test
{
public static void main(String[] args)
{
String s = " raise tech ";
s = s.trim();
System.out.println(s);
}
}
```

Q) Write a java program on join() method?

```
class Test
```

```
{
public static void main(String[] args)
{
String s = "this is java class";
s = s.join(",",s.split(" "));
System.out.println(s);
}
}
```

or

```
class Test
{
public static void main(String[] args)
{
String s = "this is java class";
s = s.join(" | ",s.split(" "));
System.out.println(s);
}
}
```